

Ivideon — документация для разработчиков

Реверс-инжиниринг приложения com.ivideon.client 3.7.1 и клиент/драйвер облачной камеры

ZigRec-Studio-iVideon · 2026-09-20

Ivideon: документация для разработчиков

Результаты реверс-инжиниринга Android-приложения **Ivideon** (`com.ivideon.client` 3.7.1) и всё необходимое, чтобы написать собственный клиент/драйвер облачной камеры Ivideon — в первую очередь **Ivideon Cute 2**.

Статус достоверности. Протокол восстановлен **статическим анализом** декомпилированного кода. Авторизованных запросов к живому API ещё не было. В тексте используются пометки **[код]**, **[вывод]**, **[гипотеза]**, **[проверено]** — см. [01-methodology.md](#), раздел 1.6. План живой проверки — в [13-open-questions.md](#).

Анализ провёл: Александр Виноградов в паре с Claude Code, 20 сентября 2026.

Теги: #Ivideon #Cute2 #реверс_инжиниринг #ReverseEngineering #Android #jadx #API5 #OAuth2 #FLV #IPкамера #IoT #Zig #ClaudeCode

Протокол за одну минуту

- 1. **Вход:** POST `https://api.ivideon.com/auth/oauth/token` — OAuth2 password grant, клиент `android-client` с пустым секретом (Basic), форма с полями устройства. В ответе — `access_token`, `refresh_token`, `api_host`, `hmac_secret`. При включённой 2FA вместо токена приходит `proceed_with_2fa` + `challenge_token`.
- 2. **API:** всё дальнейшее — на `https://{api_host}`. Стил — POST /ресурс?op=ОПЕРАЦИЯ&access_token=... с JSON-телом; ответ — `{"success": true, "result": ...}`; ошибка — не-2xx + `{"success": false, "code", "message"}`.
- 3. **Камеры:** POST `/servers?op=` с проекцией. Сервер = устройство, камера = канал; id камеры — `<serverId>:<индекс>`. Поле `features` сообщает, что умеет камера.
- 4. **Картинка:** GET `/cameras/{id}/live_preview?q=0..2` → JPEG.
- 5. **Видео:** GET `/cameras/{id}/live_stream?q=...&video_codecs=h265,h264&audio_codecs=...` → **FLV поверх HTTPS**. Архив — `archive_stream` со `start_time` в миллисекундах; перемотка = новый URL.
- 6. **Подпись:** URL из п. 4–5 подписываются: `cseq=<сессия>:<счётчик>`, `cs = hex( HMAC_SHA1(hmac_secret, hex(SHA1("GET:<path>:<query>")) ) )`.
- 7. **Камера и телефон напрямую не общаются** — только через облако. Привязка: сервер генерирует QR-код с параметрами Wi-Fi, камера читает его объективом.

Содержание

№	документ	о чём
1	Методика	объект исследования, инструменты, воспроизведение декомпиляции, как читать обфусцированный код, словарь аннотаций, уровни достоверности
2	Архитектура	схема «клиент — облако — камера», хосты и роль <code>api_host</code> , партнёрские облака, слои приложения, push
3	Авторизация	OAuth-запрос, модель токена, обновление, двухфакторный вход, отзыв, хранение, псевдокод
4	Соглашения API	стил <code>?op=</code> , способы передачи токена, заголовки, конверт, три формы списков, проекции, секунды и миллисекунды , таймауты, редиректы
5	Подпись URL	алгоритм <code>cseq / cs</code> , подводные камни кодирования, эталонный код на Python, тестовые векторы

№	документ	о чём
6	Справочник API	все эндпоинты с полями: серверы, камеры, архив, экспорт, таймлапсы, события, пользователь, пакетные запросы; минимальный набор для драйвера
7	Видео, превью, звук	FLV-поток, нативный плеер и его JNI, FFmpeg-сборка, логика воспроизведения и переключения, WebSocket-превью, голос (Opus)
8	Управление камерой	PTZ, плагины (детекторы, LED, ИК), свойства камеры, SD-карта, операции с устройством
9	Привязка камеры	QR / серийный номер / MAC, токен привязки, опрос, перевод на Wi-Fi, отсутствие LAN-обмена, особенность Cute 2
10	Ошибки	формат, алгоритм классификации, таблица кодов, ошибки OAuth, рекомендации
11	Карта кода	какой обфусцированный класс за что отвечает; что не декомпилировалось
12	Клиент на Zig	состояние <code>zig-client/</code> , команды, модули, проектные решения, ожидаемые проблемы
13	Открытые вопросы	что проверено, что нет, безопасный план живой проверки, расхождения с ранней вики

С чего начать

задача	читать
написать клиент на любом языке	3 → 4 → 5 → 6 (§6.1, §6.8) → 7
продолжить <code>zig-client</code>	12 → 3 → 4 → 5
перепроверить вывод по коду	1 → 11
провести первую живую проверку	13 (§13.4)
разобраться с привязкой новой камеры	9

PDF-версия

Вся документация одним файлом — `docs/ivideon-developer-docs.pdf` (свёрстана в LaTeX). Первоисточник — Markdown-файлы этого каталога; PDF пересобирается из них одной командой из корня проекта:

```
trac\venv\Scripts\python.exe docs\latex\build.py
```

Скрипт склеивает главы, прогоняет их через pandoc с фильтром `docs/latex/filters.lua` и компилирует `docs/latex/main.tex` движком Tectonic. Каталог `docs/latex/build/` — промежуточные файлы, его можно удалять.

PDF кликабелен. Кроме оглавления, списка иллюстраций и закладок, ссылками становятся:

что в тексте	куда ведёт
«глава 3», «раздел 3.4», «§13.4»	на главу или раздел
путь к исходнику — <code>p000/g58.java:396-467</code> , а также короткое имя класса — <code>g58</code> , <code>C2199ku</code>	на файл в <code>decompiled-ivideon-3.7.1/sources/</code>
любой существующий файл проекта — <code>zig-client/src/sign.zig</code>	на этот файл
URL без подстановок — <code>https://api.ivideon.com</code>	на этот адрес
номер тикета — <code>#27</code>	в локальный Trac (должен быть запущен)

что в тексте	куда ведёт
хэштег под заголовком главы и на титуле	в «Указатель хэштегов» в конце, а оттуда — в главы с этим тегом
«↑ Оглавление» внизу каждой страницы	к оглавлению

Кликабельный код набран синим, обычный — коричневым. Ссылки на файлы — относительные: они работают, пока PDF лежит в `docs/` внутри проекта; при открытии файла просмотрщик может спросить подтверждение.

Хэштеги. Строка `**Теги:** #тег1 #тег2` под заголовком главы (и в этом файле — общий набор для титула) при сборке превращается в кликабельные хэштеги; указатель в конце PDF и ключевые слова в свойствах файла собираются из этих строк автоматически.

Инфографика. Рисунки — TikZ-файлы в `docs/latex/diagrams/`. Место рисунка в тексте задаёт HTML-комментарий, невидимый при обычном просмотре Markdown: `←!— figure: имя | подпись →`. Диаграммы mermaid при сборке заменяются своими TikZ-двойниками — при изменении диаграммы в Markdown двойника нужно поправить вручную.

Структура проекта

путь	содержимое
<code>docs/</code>	эта документация
<code>apk/</code>	оригинальный ХАПК 3.7.1 и распакованные сплиты (<code>харк/</code>)
<code>decompiled-ivideon-3.7.1/</code>	вывод jadx: <code>sources/</code> , <code>resources/</code> — основной объект анализа
<code>decompiled/</code> , <code>uptodown-com.ivideon.client.apk</code>	установщик магазина Uptodown, не Ivideon — мусор, можно удалить
<code>docs/latex/</code>	LaTeX-оформление, фильтр pandoc, диаграммы TikZ и скрипт сборки PDF
<code>tools/</code>	портативные JRE 21, jadx 1.5.6, Zig 0.16.0, pandoc 3.11, Tectonic 0.17.0
<code>zig-client/</code>	собственный клиент (готова подпись URL с тестами)
<code>trac/</code>	локальный Trac: задачи, эпики, вики (<code>http://127.0.0.1:8000/</code> , запуск <code>trac/start-trac.bat</code>)

Трекер задач и рабочие заметки ведутся в Трас (на русском); вики-страницы `Protocol` и `ZigClient` — краткие предшественники этой документации. Номера вида `#27` в тексте — тикеты Trac.

Рамки

Исследование ведётся для совместимости с собственной камерой и собственным аккаунтом. В документации нет учётных данных и токенов; все идентификаторы в примерах вымышлены. `android-client` — публичная константа приложения. Перед живыми проверками прочтите §13.4: единичные запросы, сначала только чтение.

1. Методика реверс-инжиниринга

Теги: #реверс_инжиниринг #методика #jadx #R8 #обфускация #Retrofit #Gson

Как получены исходные данные, как воспроизвести декомпиляцию и как читать обфусцированный код. Этот документ нужен, чтобы любой вывод из остальных разделов можно было перепроверить по коду.

1.1. Объект исследования

Приложение	Ivideon для Android, пакет <code>com.ivideon.client</code>
Версия	3.7.1 (<code>version_code</code> 3317), <code>minSdk</code> 29, <code>targetSdk</code> 35
Источник	XAPK с APKPure: <code>apk/com.ivideon.client-3.7.1-apkpure.xapk</code>
Состав XAPK	<code>com.ivideon.client.apk</code> (base, 17,3 МБ), <code>config.arm64_v8a.apk</code> (нативные библиотеки, 15,9 МБ), <code>config.xxhdpi.apk</code> (ресурсы)
Обфускация	R8, map-id <code>ae391790d39fdcf0500d736c71fa099e0f969a7a28f208fca120cb0c8f9eafe8</code> (виден в заголовке каждого декомпилированного файла)

Ловушка. Файл `uptodown-com.ivideon.client.apk` в корне проекта — это **не Ivideon**, а установщик магазина Uptodown 7.39 (кнопка «Скачать» на Uptodown отдаёт свой магазин). Его декомпиляция лежит в `decompiled/` и для проекта бесполезна. Рабочий каталог — только `decompiled-ivideon-3.7.1/`.

1.2. Инструменты

Всё портативное, лежит в `tools/`, в систему ничего не ставится.

инструмент	версия	путь	назначение
Eclipse Temurin JRE	21.0.12	<code>tools/jre</code>	среда для jadx
jadx	1.5.6	<code>tools/jadx</code>	DEX → Java, ресурсы, манифест
Zig	0.16.0	<code>tools/zig</code>	сборка клиента (<code>zig-client/</code>)
Python (venv Trac)	—	<code>trac/venv/Scripts/python.exe</code>	вспомогательные скрипты, эталонные векторы подписи
pandoc	3.11	<code>tools/pandoc</code>	Markdown → LaTeX при сборке PDF этой документации
Tectonic	0.17.0	<code>tools/tectonic</code>	LaTeX-движок (XeTeX) для сборки PDF; пакеты и шрифты докачивает сам

Нативный код (`libivplayerjni.so`) дизассемблером пока не разбирался — задача Trac #12, инструмент (Ghidra или rizin) не выбран (#5).

1.3. Как воспроизвести декомпиляцию

```
:: 1. XAPK — это zip. Распаковать в apk\apk\
:: 2. Декомпилировать base APK
set JAVA_HOME=%CD%\tools\jre
```

```
tools\jadx\bin\jadx.bat --deobf --show-bad-code -d decompiled-ivideon-3.7.1
apk\xapk\com.ivideon.client.apk
```

- `--deobf` — заменяет короткие/конфликтующие имена на уникальные вида `C2199ku`, `InterfaceC2236lu`, `m14706Q`, `f26461g`. **Имена детерминированы**: при той же версии `jadx` и тех же опциях получатся те же имена, поэтому ссылки вида `p000/C2199ku.java` в документации стабильны. При другой версии `jadx` имена «поедут» — ориентируйтесь тогда на комментарий `/* renamed from: ku */` (это настоящее имя класса в DEX) и на строковые литералы.
- `--show-bad-code` — оставляет в выводе методы, которые декомпилировались с ошибками. Таких 223 из 11 746 классов (лог: `tools/jadx-ivideon.log`). В тексте они помечены `JADX WARN`/`JADX ERROR`; некоторые выведены только заглушкой `Method dump skipped` (пример — `C2199ku.m14118a`). Для них нужен просмотр `smali`/байткода (`jadx --comments-level debug` или `jadx-gui`).

Результат: `decompiled-ivideon-3.7.1/sources/` (Java) и `decompiled-ivideon-3.7.1/resources/` (манифест, `res/`, `assets`).

1.4. Структура декомпилированного кода

каталог в <code>sources/</code>	что там
<code>p000/</code>	почти весь код приложения после R8: классы без пакета, имена 2–3 символа. Здесь сервисы API, DTO, интерцепторы, репозитории, ViewModel-и
<code>com/ivideon/client/</code>	то, что R8 не переименовал: Activity/Fragment/Service из манифеста, UI (<code>p009ui/player</code> , <code>p009ui/wizard</code>)
<code>com/ivideon/sdk/network/data/error/</code>	иерархия ошибок API — имена сохранены (65 классов + <code>export/</code> , <code>timelapse/</code>)
<code>com/ivideon/sdk/player/p014iv/core/IvPlayer.java</code>	JNI-обёртка нативного плеера — имена <code>native</code> -методов сохранены (иначе не слинкуется)
<code>okhttp3/</code> , <code>retrofit2/</code> , <code>kotlin/</code> , <code>kotlinx/</code> , <code>androidx/</code> , <code>coil3/</code> , <code>io/</code> , <code>ru/</code> ...	сторонние библиотеки; частично переименованы, часть их классов «уехала» в <code>p000/</code>

Сегменты пакетов вида `p009ui`, `p014iv`, `p012v5` — это `jadx` переименовал `ui`, `iv`, `v5` (слишком короткие имена). Реальные пакеты: `com.ivideon.client.ui`, `com.ivideon.sdk.player.iv.core`, `com.ivideon.sdk.network.data.v5`.

1.5. Как читать обфусцированный код

Что переживает обфускацию

R8 переименовывает классы, методы и поля, но **не трогает**:

1. **Строковые литералы** — пути API (`"/cameras/{cameraId}/live_stream"`), JSON-ключи (`"access_token"`), лог-сообщения (`"Update token: refresh (1) starting ..."`), имена алгоритмов (`"HmacSHA1"`). Это главный инструмент навигации: поиск по строке почти всегда выводит на нужный класс.
2. **Значения аннотаций** — пути Retrofit и имена JSON-полей Gson лежат в аннотациях как строки.
3. **Имена `native`-методов и классов из манифеста**.
4. **Ссылки на методы Kotlin** — при компиляции `::signWithSecret` в байткод попадает исходное имя и сигнатура. Так восстановлено имя `signWithSecret(Ljava/lang/String;)Ljava/lang/String;` (`p000/C2470s2.java:22`).

«Классы-швейцарские ножи»

R8 склеивает много мелких лямбд и классов в один (class merging). Такой класс имеет поле-дискриминатор `int` (обычно `f...a`) и `switch` по нему в каждом методе. Примеры: `g58`, `kr6`, `qv5`, `C2504t`. В них **99 % кода не относится к Ivideon** (Compose, CameraX, Firebase, protobuf...). Читать надо только нужную ветку:

класс	нужная часть	как найти
<code>p000/g58.java</code>	метод <code>m07583n</code> — интерцептор авторизации и подписи	создаётся как <code>new g58(7, i85Var)</code> в <code>C2199ku</code>
<code>p000/kr6.java</code>	конструктор <code>kr6(C2504t)</code> (строки 626–647) — генерация идентификатора сессии	<code>this.f26374a = 7</code>
<code>p000/qv5.java</code>	конструктор <code>qv5(i85, ry1)</code> (697–701) и метод <code>m17621g</code> (358–366) — форма OAuth	<code>this.f36623a = 6</code>
<code>p000/C2504t.java</code>	<code>case 1</code> (119–138) — <code>signWithSecret</code> , HMAC-SHA1	по строке <code>"HmacSHA1"</code>

Словарь аннотаций сетевого слоя

Сетевой слой — Retrofit 2 + OkHttp + **Gson**. Аннотации Retrofit и собственные аннотации Ivideon переименованы; соответствие восстановлено по их форме (наличие `value()`, `encoded()`, `alternate()`) и по использованию.

обфусц.	оригинал	смысл
<code>@wy6("...")</code>	<code>retrofit2.http.POST</code>	метод POST, путь
<code>@mg4("...")</code>	<code>retrofit2.http.GET</code>	метод GET, путь (пустой — если URL динамический)
<code>@g47("x")</code>	<code>retrofit2.http.Path</code>	подстановка в <code>{x}</code>
<code>@fp7("x")</code>	<code>retrofit2.http.Query</code>	query-параметр; <code>null</code> — параметр опускается
<code>@ey0</code>	<code>retrofit2.http.Body</code>	тело запроса, сериализуется Gson в JSON
<code>@iba</code>	<code>retrofit2.http.Url</code>	полный URL передаётся аргументом
<code>@xa4</code>	<code>retrofit2.http.FormUrlEncoded</code>	тело <code>application/x-www-form-urlencoded</code>
<code>@zz3</code>	<code>retrofit2.http.FieldMap</code>	карта полей формы (есть <code>encoded()</code>)
<code>@bo4</code>	<code>retrofit2.http.HeaderMap</code>	заголовки аргументом (есть <code>allowUnsafeNonAsciiValues()</code>)
<code>@ok8("x")</code>	<code>com.google.gson.annotations.SerializedName</code>	имя JSON-поля (есть <code>alternate()</code>)
<code>@mn6</code>	собственная Ivideon	публичный запрос — токен не добавляется
<code>@zc0</code>	собственная Ivideon	токен в заголовке <code>Authorization: Bearer</code>
<code>@l85</code>	собственная Ivideon	стримовый URL — подписывается (<code>cseq</code> , <code>cs</code>)
<code>@kr7</code>	собственная Ivideon	метод возвращает <code>String</code> — «сырой» JSON, который разбирается вручную выше по стеку

Прочие типы: `ek6<T>` — обёртка вызова (аналог `retrofit2.Call<T>` с колбэками статуса), `sn6` — пустой результат-синглтон («данных нет, важен только успех»), `v28` — `okhttp3.ResponseBody` (бинарный ответ), `ct5` — DTO `LivePreviewChannel{url}` (имя видно в его `toString()`), `ps4` — `okhttp3.HttpUrl`, `e82` — `HttpUrl.Builder`, `g08` — `okhttp3.Request`, `a40` — `Request.Builder`, `fo4` — `okhttp3.Headers`.

Рецепты поиска

задача	приём
найти все эндпоинты	grep по <code>@wy6\(" и @mg4\(" в p000/</code> — их объявляют только интерфейсы сервисов
найти JSON-поля запроса/ответа	открыть класс-аргумент <code>@eu0</code> или параметр типа <code>ek6<...></code> , читать <code>@ok8("...")</code>
найти, кто вызывает эндпоинт	grep по имени метода с префиксом <code>m</code> и номером, например <code>m14706Q\("</code>
найти значения enum	открыть enum; у «строковых» enum значение — аргумент конструктора (<code>UPC("up")</code>), а <code>toString()</code> его и возвращает
найти, куда уходит значение поля DTO	grep по геттеру (<code>m7191e\(" для <code>api_host</code></code>)
понять ветку класса-ножа	найти <code>new <класс>(N, ...)</code> и читать <code>case N</code>

1.6. Уровни достоверности

Во всей документации используются пометки:

пометка	значение
[код]	прямо видно в декомпилированном коде; указаны файл и строки
[вывод]	логическое следствие из кода, но не дословно написано в нём
[гипотеза]	предположение; нужно проверить
[проверено]	подтверждено реальным запросом к серверу или запуском

На момент написания **[проверено]** только: редиректы хостов (`api.ivideon.com` → `eu01-api.ivideon.com`) и совпадение реализации подписи в `zig-client/src/sign.zig` с независимой реализацией на Python. **Ни один авторизованный запрос к живому API ещё не выполнялся** (Trac #20). Поведение сервера (коды ошибок, точные формы ответов, лимиты) известно только в той мере, в какой его отражает код клиента.

1.7. Правовые и этические рамки

Исследование ведётся для совместимости с **собственной** камерой и **собственным** аккаунтом пользователя. Документация не содержит учётных данных, токенов и способов обхода чужой авторизации. `client_id android-client` с пустым секретом — публичная константа из приложения, а не секрет. Условия использования сервиса Ivideon и ограничения по частоте запросов не изучены (Trac #36) — при живых проверках не создавайте нагрузку: единичные запросы, без циклов опроса быстрее, чем это делает само приложение.

2. Архитектура: облако, хосты, устройство приложения

Теги: #архитектура #облако #api_host #API5 #хосты #push

2.1. Общая схема

```
flowchart LR
    subgraph Клиент
        APP[Приложение / свой драйвер]
        PL[Плеер FLV]
    end
    subgraph Облако Ivideon
        AUTH[auth-хост<br/>api.ivideon.com]
        API[региональный API5<br/>https://api_host]
        MEDIA[медиа: FLV, JPEG,<br/>WebSocket-превью, голос]
    end
    CAM[Камера<br/>Ivideon Cute 2]

    APP -- "1. POST /auth/oauth/token" --> AUTH
    AUTH -- "токен: access_token, api_host, hmac_secret" --> APP
    APP -- "2. POST /...?op=...&access_token=" --> API
    APP -- "3. подписанный URL" --> PL
    PL -- "GET live_stream / archive_stream" --> MEDIA
    CAM == "исходящее соединение<br/>(протокол прошивки, в APK не виден)" ==> API
    API -. "команды: PTZ, плагины, Wi-Fi" .-> CAM
```

Ключевые свойства:

- Камера сама держит соединение с облаком. Приложение никогда не обращается к камере напрямую — ни при просмотре, ни при привязке (09-camera-attachment.md). Команды (PTZ, настройки, скан Wi-Fi) идут в облако, а оно передаёт их камере; отсюда ошибки CAMERA_OFFLINE, DEVICE_TIMED_OUT.
- Весь клиентский трафик — HTTPS к облаку: JSON-API, FLV-видео, JPEG, WebSocket-превью, загрузка голоса.
- Протокол «камера ↔ облако» находится в прошивке и из Android-приложения не восстанавливается.

Для драйвера это означает: без облака Ivideon (и без интернета) этим путём картинку не получить. Локальный доступ возможен, только если прошивка Cute 2 сама открывает RTSP/ONVIF/HTTP — это проверяется сканом камеры (Трас #32), а не анализом приложения.

2.2. Хосты

Значения по умолчанию — p000/l21.java:47-51; логика выбора — p000/k85.java:47-151.

роль	значение по умолчанию	для чего
auth	https://api.ivideon.com	POST /auth/oauth/token, POST /idam?op=SHARE
основной API	https://{token.api_host}	все авторизованные вызовы, включая URL видеопотоков и превью
public API5	https://api.ivideon.com (или api5_host облака default)	публичные вызовы: регистрация, сброс пароля, баннеры
2FA	https://{api5_host} из ответа-вызова	шаги двухфакторного входа

роль	значение по умолчанию	для чего
customer clouds	https://api.ivideon.com	POST /customer_clouds?op=FINDD
streaming	https://streaming.ivideon.com	не используется: читается один раз и нигде не применяется — наследие старых версий [вывод]
веб-портал	https://go.ivideon.com	ссылки «тариф», «оплата», «помощь»; App Links

api_host — главный факт

Поле api_host из ответа с токеном становится базовым URL Retrofit для основного сервиса (InterfaceC2236lu) и сервиса картинок (InterfaceC2345or): p000/yl6.java:28 → p000/C2199ku.java:48–74 . Если в значении нет схемы, добавляется https:// (p000/or7.java:167–171). **Видеопотоки идут на тот же хост** — отдельного стримингового домена в 3.7.1 нет.

[проверено]: api.ivideon.com редиректит на eu01-api.ivideon.com — вероятно, это и есть типичное значение api_host для европейского региона.

Это закрывает вопрос Trac #14 («где клиент подставляет api_host и когда используется streaming »).

Партнёрские облака и on-premise

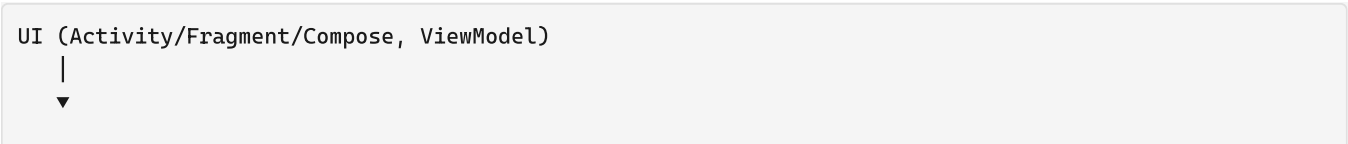
- POST https://api.ivideon.com/customer_clouds?op=FINDD — без авторизации и тела; результат — список {id, id_prefix, id_prefix_range[], geo_host, geo_host_alt, api4_host, api5_host} (p000/qm2.java, p000/xz1.java). На экране входа пользователь может выбрать облако; тогда auth-хост = https://{api4_host}, public API5 = https://{api5_host} . В приложении защиты бренды Ivideon и Nobelic .
- «Microcloud» — адрес собственного сервера (протокол://хост:порт) вводится вручную и подменяет все хосты.

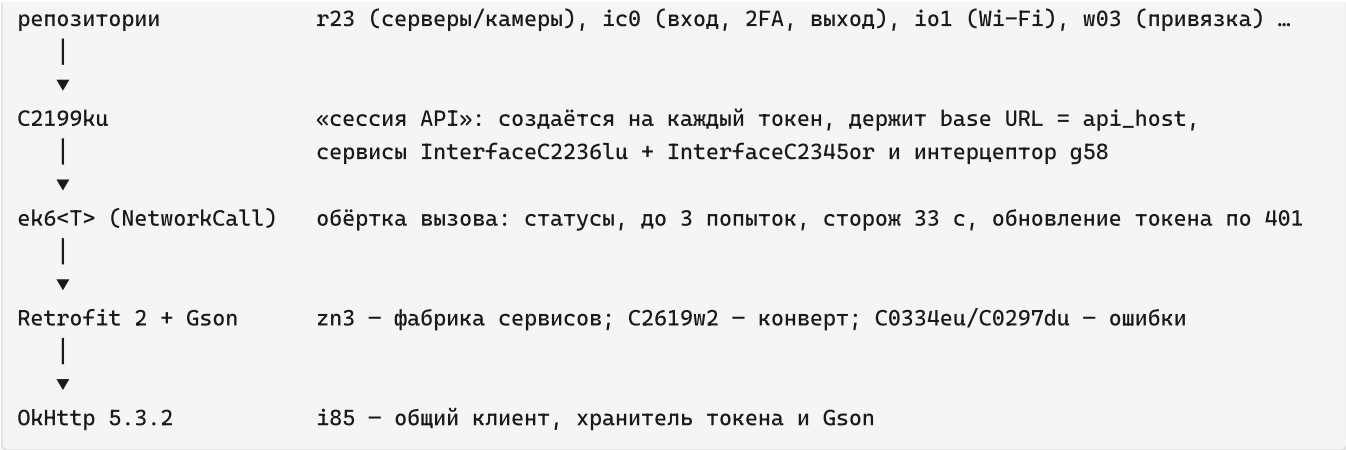
Обычному пользователю Ivideon ни то ни другое не нужно.

2.3. Поколения API

поколение	признак	статус в 3.7.1
API5	?op=, конверт {success, result}, строковые коды ошибок	всё приложение
API4	конверт {success, response}, числовые коды, параметр sessionId	сервис создаётся и сразу отбрасывается — мёртвый код; остался адаптер ошибок
Open API (openapi-alpha.ivideon.com)	—	в приложении не используется; публичной документации не найдено (Trac #19: openapi-docs.ivideon.com — пустая заглушка)

2.4. Устройство приложения (сетевой слой)





класс	роль
p000/i85.java	ядро SDK: OkHttpClient, Gson, текущий токен, проверка срока, блокировка обновления
p000/k85.java	сборка i85: выбор хостов по режиму (обычный / облако партнёра / microcloud)
p000/y16.java	набор сервисов, привязанных к токenu; пересоздаётся при смене токена
p000/ob0.java	хранение токена (SharedPreferences PREFSES / ACCESS_TOKEN), состояние «вошёл/вышел»
p000/ic0.java	сценарии входа, 2FA, выхода, отзыва токена
p000/h85.java	конечный автомат обновления токена
p000/e21.java	выполнение вызова: повторы, разбор ошибок, реакция на 401
p000/cv0.java	сборщик пакетных запросов

Полная карта — 11-code-map.md.

2.5. Прочие каналы приложения

канал	назначение	нужно драйверу?
Push: FCM, HMS (Huawei), RuStore	уведомления о событиях; подписка — POST /push_subscriptions?op=CREATE	нет — привязано к экосистемам Android; замена — опрос events?op= FIND
AppMetrica (Яндекс), Firebase Crashlytics/Performance, OK Tracer	аналитика и сбор сбоев	нет
ML Kit barcode	сканер наклейки камеры и QR	нет

Формат push-уведомлений (справочно)

Data-сообщения; плоская карта строк превращается в JSON (p000/jrc.java), server_id + camera_id склеиваются в id камеры. Событие камеры: type, event, time, title, description, camera_id, camera_name, preview?, chain_id? (p000/nc1.java). Типы: motion, sound, sensor-event, thermal, network-status (camera-online/offline, server-online/offline), doorbell-call (started/finished), analytics-{line-crossing, area-invasion, tampering, loitering, object-removal}, а также face-recognized, marketing, billing, system (camera-shared, camera-archive-export-completed, timelapse-export-completed). Эти же имена — ориентир для значений type в events?op= FIND [гипотеза: там имена иерархические, через /].

2.6. Манифест (кратко)

`minSdk 29, targetSdk 35, allowBackup=false`. Deep links: `ivideon://start`, `ivideon://signin`; App Links `https://go.ivideon.com/live/`, `/archive/`, `/camera-settings/` объявлены, но Activity не читают `getData()` — фактически не работают **[вывод]**. Экраны отладки (`DebugSettingsController`, `ConfigurationController`) присутствуют в релизной сборке; через них, в частности, переопределяются хосты (`SharedPreferences` `ApiServerUrls`).

3. Авторизация и жизненный цикл токена

Теги: #OAuth2 #токен #refresh_token #двухфакторка #безопасность

OAuth2 password grant с публичным клиентом `android-client`, обновление по `refresh_token`, двухфакторный вход, отзыв токена.

Достоверность: запросы, поля и логика клиента — **[код]**. Ответы сервера известны только по моделям, в которые клиент их разбирает. **Ни один вход на живом сервере не выполнялся** (Trac #20).

3.1. Получение токена

Источники: `p000/jc0.java` (сервис), `p000/qv5.java:358-366, 697-701` (форма), `p000/C2552ua.java:63-81` (Basic-заголовок).

```
POST https://api.ivideon.com/auth/oauth/token
Authorization: Basic YW5kcm9pZC1jbGllbnQ6
Content-Type: application/x-www-form-urlencoded

grant_type=password&username=<email>&password=
<пароль>&client_type=android&client_version=3.7.1&device_instance_id=<id>&device_type=
<строка>&device_name=<строка>&trusted_device=true
```

`YW5kcm9pZC1jbGllbnQ6` = `base64("android-client:")` — `client_id` `android-client`, **секрет пустой**. Это публичная константа приложения.

поле формы	значение в приложении	что слать своему клиенту
<code>grant_type</code>	<code>password</code>	то же
<code>username</code> , <code>password</code>	логин (email) и пароль	то же
<code>client_type</code>	<code>android</code>	<code>android</code> — с другим значением поведение сервера неизвестно
<code>client_version</code>	<code>3.7.1</code>	<code>3.7.1</code>
<code>device_instance_id</code>	Firebase Installation ID — 22 символа <code>base64url</code> (<code>p000/q10.java:21-27</code>)	любой стабильный идентификатор установки: сгенерировать один раз и хранить рядом с токеном
<code>device_type</code>	"<MANUFACTURER> <MODEL>, Android <RELEASE>", напр. <code>samsung SM-G991B, Android 13</code> (<code>p000/p52.java:111</code>)	произвольная строка-описание
<code>device_name</code>	имя устройства из настроек либо <code>Build.MODEL</code>	произвольная строка
<code>trusted_device</code>	всегда <code>true</code>	<code>true</code>

[гипотеза] `device_instance_id` + `trusted_device` влияют на то, будет ли сервер запрашивать второй фактор повторно с того же устройства. Меняя идентификатор при каждом входе, вы, вероятно, будете каждый раз выглядеть как новое устройство.

Хост: в обычном режиме — `https://api.ivideon.com`; для партнёрского облака — `https://{api4_host}` этого облака (см. [02-architecture.md](#)).

3.2. Ответ: три варианта

A. Токен

Модель `p000/fr7.java` (+ `p000/C2470s2.java`):

поле	тип	примечание
<code>access_token</code>	string	он же — идентификатор токена для отзыва
<code>token_type</code>	string	
<code>expires_in</code>	int, секунды	
<code>refresh_token</code>	string?	если нет — обновление невозможно
<code>created_at</code>	дата?	если отсутствует, клиент подставляет локальное «сейчас»
<code>api_host</code>	string	базовый хост всего дальнейшего API ; если без схемы — клиент добавляет <code>https://</code> (<code>p000/or7.java:167–171</code>)
<code>hmac_secret</code>	string?	ключ подписи стримовых URL; пусто → URL не подписываются (05-url-signing.md)
<code>owner_id</code>	long	id пользователя; подставляется в <code>user</code> , <code>/users/{uid}</code>
<code>owner_type</code> , <code>scope</code> , <code>client_type</code> , <code>client_version</code>	string	
<code>limited_to_2fa</code>	bool?	<code>true</code> → токен ограничен: годится только для настройки 2FA (3.5)

Ответ приходит **без конверта API5** — голый JSON-объект.

B. Вызов второго фактора — HTTP 200, не ошибка

```
{
  "proceed_with_2fa": true,
  "message": "...",
  "challenge_token": "...",
  "api5_host": "...",
  "available_methods": [
    {
      "id": "...",
      "type": "email" | "sms",
      "value": "...",
      "owner_id": 123,
      "state": {
        "no_resend_until": ...,
        "resend_timeout": ...,
        "submit_attempts": ...
      }
    }
  ]
}
```

Источники: `p000/j5a.java`, `p000/bb0.java`, `p000/eb0.java`, адаптер `p000/C2270mr.java` (восстановлен из fallback-дампа). Дальше — раздел 3.4.

Ранняя версия вики предполагала, что признак 2FA — поле `limited_to_2fa` в токене. Это **другой** сценарий (3.5). Обычный вход с включённой 2FA даёт ответ `proceed_with_2fa`.

C. Ошибка OAuth

`{"error", "error_description", "reason", "retry_in"}` — см. [10-errors.md](#), раздел 10.4. Неверный пароль → `AuthError`; `reason=T00_MANY_LOG_IN_ATTEMPTS` → ждать `retry_in`.

3.3. Срок жизни и обновление

аспект	поведение приложения	источник
проверка срока	токен считается истёкшим при <code>now > created_at + expires_in - 300 с</code> — запас 5 минут	<code>p000/i85.java:218-240</code>
упреждающее обновление	при старте приложения и перед сборкой каждого URL потока	<code>App.java:293</code> , <code>PlayerController.java:803,831</code>
реактивное обновление	HTTP 401 → принудительный refresh → повтор исходного запроса	<code>p000/e21.java:86-103</code>
исключение	401 с <code>message = "Session's subject doesn't exist."</code> → «аккаунт деактивирован или токен отозван», refresh не делается	<code>AuthError.java:20</code> , <code>e21.java:89-92</code>
конкурентность	один <code>AtomicBoolean</code> : параллельные попытки не шлют второй запрос, а подписываются на идущий	<code>i85.java:243-295</code>
неудача с <code>AuthError</code>	состояние <code>NO_FATE</code> → токен сбрасывается, пользователь разлогинен	<code>p000/h85.java:59-81</code>
прочая неудача (сеть)	<code>FAILED</code> , токен сохраняется	там же
запасной вход паролем	нет — пароль не хранится (лог: <code>we don't keep credentials anymore</code>)	<code>h85.java:90</code>

Запрос обновления — тот же эндпоинт и тот же Basic-заголовок (`h85.java:113-118`):

```
grant_type=refresh_token
refresh_token=<refresh_token>
access_token=<текущий access_token>          ← нестандартно для OAuth2, но приложение шлёт
client_type, client_version, device_instance_id, device_type, device_name, trusted_device=true
```

После обновления приложение **пересоздаёт все сервисы** с новым `api_host` и заново генерирует `session` для подписи URL.

3.4. Вход с двухфакторной авторизацией

Все шаги — публичные запросы (без `access_token`) на хост `https://{api5_host}` из ответа-вызова; авторизует их `challenge_token`. Источники: `p000/tl7.java`, `p000/sl7.java:23-34`, `p000/ic0.java:581, 645, 715`.

```
sequenceDiagram
    participant A as Клиент
    participant O as auth-хост
    participant P as api5_host
    A->>O: POST /auth/oauth/token (password)
    O->>A: 200 {proceed_with_2fa, challenge_token, api5_host, available_methods}
    Note over A: методов несколько — выбирает пользователь
    A->>P: POST /two_factor_methods/{methodId}?op=SELECT {challenge_token}
    P->>A: {no_resend_until, submit_attempts}
    Note over P: сервер отправляет код на email/SMS
    opt код не пришёл
        A->>P: POST /two_factor_methods/{methodId}/code?op=RESEND {challenge_token}
    end
```

A→P: POST /two_factor_methods/{methodId}/code?op=SUBMIT {challenge_token, code}
 P→A: полноценный токен (модель из 3.2 A)

шаг	запрос	тело	ответ
выбрать метод	POST /two_factor_methods/{id}? op=SELECT	{"challenge_token"}	{no_resend_until, submit_attempts}
переслать код	POST /two_factor_methods/{id}/code? op=RESEND	{"challenge_token"}	то же
отправить код	POST /two_factor_methods/{id}/code? op=SUBMIT	{"challenge_token", "code"}	токен

Ответы, предположительно, в конверте API5 **[вывод: общий конвертер]**. Ошибки: CHALLENGE_NOT_FOUND, CHALLENGE_EXPIRED, TWO_FACTOR_METHOD_NOT_FOUND, TWO_FACTOR_METHOD_NOT_SELECTED, TOO_FREQUENT (соблюдайте no_resend_until), INVALID_CODE, NO_ATTEMPTS_LEFT, BAD_CODE, OTP_LOCKED.

3.5. Принудительная настройка 2FA (limited_to_2fa)

Если сервер требует 2FA, а она не настроена: либо ошибка OAuth invalid_grant / reason=2FA_IS_MANDATORY, либо токен с limited_to_2fa: true. С ограниченным токеном приложение сразу ведёт на привязку телефона (signin/C0237c.java:153–162):

запрос (на api_host, с access_token)	тело
POST /two_factor_methods?op=CREATE	{"value": "<телефон>", "type": "sms", "password": "<пароль>"}
POST /two_factor_methods?op=CONFIRM	{"two_factor_method_id": "...", "code": "..."}>

Каким вызовом ограниченный токен превращается в полный — не прослежено; **[вывод]**: приложение входит заново и проходит сценарий 3.4.

Минимальная стратегия для своего клиента: распознать proceed_with_2fa и limited_to_2fa, сообщить пользователю и не пытаться работать с таким ответом как с токеном.

3.6. Выход и отзыв токена

POST https://{api_host}/access_tokens/{accessTokenId}?op=REVOKE

- публичный (@mn6): access_token в query **не** добавляется, тела нет;
- accessTokenId — **сама строка access_token** (p000/ic0.java:180–183, 345–348; косвенно подтверждается десериализатором p000/C2582v2.java, где id токена отображается на access_token);
- ошибки отзыва приложение игнорирует; затем очищает локальные данные.

3.7. Хранение

Приложение хранит токен **в открытом виде**: SharedPreferences PREFSES, ключ ACCESS_TOKEN, JSON модели токена (p000/ob0.java:79–84). Без шифрования и без AccountManager. Пароль не хранится.

Для своего клиента: access_token, refresh_token и hmac_secret — секреты одного уровня с паролем на время жизни. Файл — только для текущего пользователя ОС, вне репозитория (.gitignore), не печатать в логах. Учтите, что само приложение передаёт access_token в query-строке URL (включая ссылки на веб-портал go.ivideon.com, параметр _go.session) — URL с токеном нельзя публиковать.

3.8. Регистрация и сброс пароля (для полноты)

Публичные вызовы на public-API5-хосте (p000/tl7.java):

запрос	тело
POST /users?op=CREATE	login, password, language, country, device_type, device_name, device_instance_id, sign_in, marketing_data, account_type + константы client_id: "android-client", client_type: "android", client_version: "3.7.1", partner_id: "ivideon" (p000/xt8.java). Ответ — пользователь с встроенными полями токена
POST /users?op=RESET_PASSWORD	{"login"}
POST /partners/ivideon/signup_form_config?op=GET	{"client_language", "client_type": "mobile", "ip": null}

Входа через соцсети/внешних провайдеров в приложении нет; демо-режима с отдельным токеном тоже нет.

3.9. Псевдокод клиента

```
def login(email, password):
    r = post_form(AUTH_HOST + "/auth/oauth/token", basic("android-client", ""), {
        "grant_type": "password", "username": email, "password": password, **DEVICE_FIELDS,
        "trusted_device": "true"})
    if r.status != 200:
        raise OAuthError(r.json()) # error / reason / retry_in
    j = r.json()
    if j.get("proceed_with_2fa"): return TwoFactorChallenge(j) # раздел 3.4
    if j.get("limited_to_2fa"): raise TwoFactorSetupRequired() # раздел 3.5
    j.setdefault("created_at", time.time())
    return Token(j) # base_url = ensure_https(j["api_host"])

def call(token, method, path, **kw):
    if token.expires_within(300): token = refresh(token)
    r = send(token, method, path, **kw)
    if r.status == 401 and r.message != "Session's subject doesn't exist.":
        token = refresh(token) # AuthError при refresh → требовать новый вход
        r = send(token, method, path, **kw)
    return r
```

4. Соглашения API: запросы, ответы, типы данных

Теги: #API5 #JSON #проекции #конверт_ответа #unix_time #таймауты

Правила, общие для всех вызовов «API5» — так в коде называется текущее поколение облачного API Ivideon. Достоверность — **[код]**, если не помечено иначе.

4.1. Стил

- Почти всё — **POST <ресурс>?op=<ОПЕРАЦИЯ>** с JSON-телом (`Content-Type: application/json`), даже чтение: `POST /servers?op=FIND`, `POST /cameras/{id}?op=GET`.
- Операции: `FIND`, `FIND_ONE`, `GET`, `CREATE`, `UPDATE`, `UPSERT`, `DELETE`, `SET`, `CONFIGURE`, `INVOKE`, `SHARE`, `REVOKE`, `POLL`, `STREAM`, `START`, `TEST`, `CHECK`, `FORMAT`, `SCAN`, `CONNECT`, `RECOGNIZE`, `EXPORT`, `MARK`, `SELECT`, `RESEND`, `SUBMIT`, `GET_ID`, `GET_TYPES`, `RESET_PASSWORD`...
- Настоящий **GET** — только для бинарных данных: видеопотоки и изображения.
- Исключение — `POST /auth/oauth/token: application/x-www-form-urlencoded`, ответ без конверта ([03-auth.md](#)).
- Базовый URL авторизованных вызовов — `https://{api_host}` из токена ([02-architecture.md](#)).

4.2. Авторизация запроса

Интерцептор `p000/g58.java:396-467`; режим задаётся аннотацией метода сервиса.

режим	аннотация	как передаётся токен	где применяется
обычный	нет	query-параметр <code>access_token=<token></code>	все вызовы <code>?op=</code>
Bearer	@zc0	заголовок <code>Authorization: Bearer <token></code>	<code>GET /events/{id}/thumbnails</code> , загрузка картинок/клипов по готовому URL
публичный	@mn6	никак	OAuth-токен, 2FA-вход, регистрация, сброс пароля, <code>device_models?op=RECOGNIZE</code> , <code>access_tokens/{id}?op=REVOKE</code> , <code>customer_clouds?op=FIND</code>
подписанный	@l85	<code>access_token</code> в query + <code>cseq + cs</code>	<code>live_stream</code> , <code>archive_stream</code> , <code>live_preview</code> — 05-url-signing.md

Пример обычного вызова:

```
POST https://eu01-api.ivideon.com/servers?op=FIND&access_token=<token>
Content-Type: application/json

{"projection": {"id": 1, "name": 1, "online": 1, "cameras": {"id": 1, "name": 1, "online": 1}}}
```

Токен в query-строке попадает в логи прокси и серверов — это свойство API. В своих логах маскируйте `access_token=...` и `cs=...`.

4.3. Заголовки

Приложение **не добавляет собственных заголовков по умолчанию**: нет ни `X-*`, ни версии приложения, ни `Accept-Language`. `User-Agent` — стандартный для OkHttp: `okhttp/5.3.2 (p000/ns7.java:360)`. Версия и

платформа клиента передаются только полями `client_version` / `client_type` при входе; язык — полями тел (`lang`, `language`).

Нативный плеер, открывая видео-URL, по-видимому, представляется как `Boost.Beast/359` **[вывод]**; WebSocket-превью — фиксированным User-Agent Chrome 27.

Следствие: серверу не по чему отличить сторонний клиент, кроме User-Agent. Обязателен ли определённый User-Agent — **не проверено**.

4.4. Конверт ответа

Успех (`p000/C2619w2.java:64-114`):

```
{"success": true, "result": <значение>}
```

- `success` — строго boolean; `result` **обязателен даже для операций без данных** (тогда это `{}`, `"` или `null` — `p000/un6.java`);
- порядок ключей не важен; все прочие ключи верхнего уровня клиент пропускает;
- идентификатора запроса, `total`, `next`, `retry_after` на уровне конверта клиент не читает.

Ошибка — не-2xx + `{"success": false, "code", "message", "details"?}` — [10-errors.md](#).

Списки в `result`

Приложение принимает **три формы** списка (`p000/C2619w2.java:131-165`) — своему клиенту стоит принимать все:

форма	пример	обработка
массив	<code>[{...}, {...}]</code>	как есть
объект с <code>items</code>	<code>{"items": [{...}, {...}]}</code>	берётся <code>items</code>
словарь по id	<code>{"100-abc": {...}, "100-def": {...}}</code>	в каждое значение вставляется <code>"id": "<ключ>"</code> , значения становятся списком

Какую форму отдаёт сервер для конкретного вызова, покажет живая проверка; у `/servers?op= FIND` это один из главных открытых вопросов.

Пагинация

В запросах — `skip` и `limit`. Метаданных пагинации в ответах клиент не читает; конец выборки определяется, видимо, по размеру страницы меньше `limit` **[вывод]**.

4.5. Проекция

Читающие вызовы (`servers FIND/GET`, `cameras GET`, `events GET`, `users GET`, `folders FIND`, `notification_settings FIND`, `exported_records FIND`, `permission_grants FIND`) принимают `projection` — описание нужных полей в стиле MongoDB:

```
{"projection": {"id": 1, "name": 1, "cameras": {"id": 1, "online": 1}}}
```

- `1` — включить поле; вложенный объект — проекция вложенного документа;

- встречается форма исключения: `{"option": 0, "_others": 1}` — «всё, кроме `option`»;
- ключи услуг содержат `>: "notify>motion"`.

Методы с проекцией помечены `@kr7` — приложение получает `result` сырой JSON-строкой и разбирает в другом слое; поэтому их ответы типизированы в коде хуже остальных. Что вернёт сервер **без** `projection` (всё или минимум) — не проверено; безопаснее передавать проекцию всегда. Готовый образец — в [06-api-reference.md](#), раздел 6.1.

4.6. Типы данных

тип	на проводе	детали
дата (поля <code>Date</code>)	JSON-число, Unix-секунды , возможна дробная часть до мс: <code>17000000000</code> или <code>17000000000.5</code>	запись — <code>p000/C2647wu.java</code> ; чтение — <code>p000/nq2.java</code> : значение $\geq 3.1536 \cdot 10^{10}$ трактуется как миллисекунды, иначе как секунды
время в миллисекундах (поля <code>Long</code>)	целое, Unix-мс	только: <code>archive_timeline</code> (<code>start_time</code> , <code>end_time</code> , <code>duration</code>) и <code>query</code> -параметры <code>archive_stream</code>
календарная дата	строка <code>yyyy-MM-dd</code>	<code>archive_calendar</code>
<code>turned_off_until</code>	число	<code>0</code> — включена, <code>-1</code> — выключена бессрочно, иначе Unix-секунды (<code>p000/ud7.java</code>)
качество <code>q</code>	<code>0 / 1 / 2</code>	<code>low / medium / high</code>
id пользователя	в токене — число (<code>owner_id</code>), в телах и путях — строка	
id камеры	<code><serverId>:<индекс></code>	двоеточие в пути URL не кодируется
часовой пояс	IANA (<code>Europe/Moscow</code>)	
<code>null</code>	поле опускается	<code>serializeNulls</code> не включён
enum	строка в нижнем регистре (<code>"wifi"</code> , <code>"in_progress"</code>) либо число (<code>q</code> , чувствительность звука)	значения — в справочнике

Секунды или миллисекунды — главный источник ошибок: `events?op=Find` и `archive?op=EXPORT` принимают секунды, `archive_timeline` и `archive_stream` — миллисекунды.

4.7. Сеть, таймауты, повторы

Конфигурация `OkHttp` (`p000/i85.java:76–92`):

параметр	значение
TLS	1.3 / 1.2 / 1.1 / 1.0; системное хранилище сертификатов. Пиннинга нет (<code>CertificatePinner</code> по умолчанию, в <code>network_security_config.xml</code> нет <code><pin-set></code>); cleartext запрещён для <code>ivideon.com</code> и <code>extcam.com</code>
таймауты	connect 30 с, read 30 с, write 10 с; общий сторож на вызов — 33 с
редиректы	включены (по умолчанию <code>OkHttp</code>)
повторы	до 3 попыток на вызов; не повторяются 4xx, ошибки JSON, 2FA
прокси	системный

Отсутствие пиннинга означает, что трафик официального приложения можно исследовать MITM-прокси с пользовательским сертификатом (на Android ≥ 7 — с поправкой на доверие только системным CA). Это резервный способ живой проверки, если статического анализа не хватит.

О редиректах. `api.ivideon.com` отвечает редиректом на региональный хост (`eu01-api.ivideon.com`) — **[проверено]** для запроса к корню. OkHttp при 301/302 превращает POST в GET и теряет тело; при 307/308 — сохраняет. Каким кодом сервер редиректит именно `POST /auth/oauth/token`, неизвестно. Своему клиенту: **не следовать редиректам автоматически для POST** — получив 3xx, повторить тот же POST с тем же телом на `Location`.

4.8. Частота запросов

Интервалы, которые использует само приложение — разумный верхний предел и для своего клиента:

что	интервал
опрос токена привязки	3 с
опрос состояния сервера	5 с
опрос пакетного запроса	1 с
поток превью в сетке камер	1 кадр/с

Лимиты сервера неизвестны (Trac #36). Коды `TOO_MANY_LOGIN_ATTEMPTS`, `TOO_FREQUENT`, `LIMIT_EXCEEDED` показывают, что ограничения есть.

5. Подпись стримовых URL (cseq + cs)

Теги: #подпись_URL #HMAC #SHA1 #cseq #OkHttp #тестовые_векторы

Запросы к видео и превью (live_stream , archive_stream , live_preview) приложение отправляет не через свой HTTP-клиент, а **строит URL и отдаёт его плееру/загрузчику картинок**. Чтобы такой URL нельзя было подделать или переиспользовать, он подписывается секретом hmac_secret из токена.

Достоверность: алгоритм — [код]; то, что сервер действительно проверяет подпись и как реагирует на ошибку, — **не проверено**.

5.1. Когда подпись добавляется

Источник: p000/g58.java, метод mo7583n (строки 396–467).

Подпись добавляется, только если выполнены оба условия (строка 431):

- 1. метод сервиса помечен @l85. Таких методов три: * GET /cameras/{cameraId}/live_stream — p000/InterfaceC2236lu.java:78 * GET /cameras/{cameraId}/archive_stream — p000/InterfaceC2236lu.java:105 * GET /cameras/{cameraId}/live_preview — p000/InterfaceC2345or.java:12
- 2. в токене есть непустой hmac_secret (флаг C2470s2.f38388n, вычисляется в p000/C2470s2.java:21).

Если hmac_secret пуст — URL уходит только с access_token, без cseq и cs.

Исправление к ранней версии вики: live_preview — это @l85 (токен в query + подпись), а не @zc0 (Bearer). Bearer используется для GET /events/{eventId}/thumbnails и для загрузки картинок по готовому URL (InterfaceC2345or.m16309a, m16312d).

5.2. Алгоритм

```
session = 8 случайных символов из [0-9a-zA-Z]
        генерируется один раз при создании объекта токена и живёт вместе с ним
cseq    = session + ":" + counter
        counter – SystemClock.elapsedRealtime(): миллисекунды с загрузки устройства

1. К URL добавляется query-параметр access_token=<token>           (шаг общий для всех обычных запросов)
2. К URL добавляется query-параметр cseq=<cseq>
3. to_sign = METHOD + ":" + path + ":" + query + ":" + body
4. digest  = hex_lower( SHA1( utf8(to_sign) ) )                     # 40 символов
5. cs      = hex_lower( HMAC_SHA1( key = utf8(hmac_secret), msg = utf8(digest) ) )
6. К URL добавляется query-параметр cs=<cs>
```

Детали, на которых легко ошибиться:

элемент	точное значение	где в коде
METHOD	метод HTTP заглавными: GET	g08Var2.f13158b , g58.java:442
path	"/" + join("/", pathSegments) — декодированные сегменты пути OkHttp, с ведущим слэшем. Двоеточие в id камеры (100-...:0) остаётся	p000/zbc.java:40-43 , ps4.f34839f

элемент	точное значение	где в коде
	двоеточием	
query	закодированная строка запроса — подстрока URL после ? до #, как она есть в итоговом URL, уже включая access_token и cseq, но без cs	ps4.m17064d(), p000/ps4.java:89-96
body	тело запроса как строка UTF-8; для GET — пустая строка. Итоговая строка для GET заканчивается двоеточием	g58.java:445-452
разделитель	: (константа StringUtils.PROCESS_POSTFIX_DELIMITER из AppMetrica, равна ":")	g58.java:453
hex	нижний регистр, без разделителей (%02x)	c70.m2912D(bytes, "", ...)
порядок параметров	исходные параметры → access_token → cseq → cs. cs всегда последний	g58.java:416, 440, 462

Двойное хеширование — не опечатка. HMAC считается не от to_sign, а от hex-строки его SHA-1 (40 ASCII-символов).

Кодирование query. OkHttpClient при добавлении параметра (addQueryParameter) percent-кодирует набор QUERY_COMPONENT_ENCODE_SET; нетронутыми остаются только [A-Za-z0-9] и - . _ *. Обратите внимание: тильда ~ **кодируется** (%7E), хотя большинство библиотек её не трогают. Практически важно: запятая в video_codecs=h265,h264 превращается в %2C, двоеточие в cseq — в %3A, пробел — в %20. Подписывается именно закодированная форма, поэтому ваша реализация должна кодировать **так же, как OkHttpClient**, иначе cs не совпадёт. Реализация zig-client/src/sign.zig кодирует всё, кроме [A-Za-z0-9-.*], в %XX заглавными hex-цифрами — этого достаточно для всех значений, которые реально встречаются (цифры, h265, h264, токен, cseq).

Генерация session

p000/kr6.java:626-647 — восемь раз Random.nextInt(62), отображение: 0-9 → цифры, 10-35 → a-z (i + 87), 36-61 → A-Z (i + 29). Случайность не криптографическая, и от неё ничего не зависит: это просто идентификатор клиентской сессии.

Что такое counter [вывод]

Значение монотонно растёт и уникально в пределах session. Вероятное назначение на сервере — защита от повторного использования URL (nonce/последовательность). В своём клиенте используйте любой монотонный счётчик миллисекунд (например, время с запуска процесса или Unix-время в мс). Принимает ли сервер значения «из прошлого» относительно уже виденных для той же session — **неизвестно**; безопаснее никогда не уменьшать счётчик в пределах одной session.

5.3. Эталонная реализация (Python)

```
import hashlib, hmac, random, string, time
from urllib.parse import quote

SAFE = "-._*" # вместе с [A-Za-z0-9] — то, что OkHttpClient не кодирует в query

def enc(s: str) → str:
    # пробел → %20, ',' → %2C, ':' → %3A; quote() никогда не кодирует '~', а OkHttpClient кодирует
    return quote(s, safe=SAFE).replace("~", "%7E")

def new_session() → str:
```

```

    return "".join(random.choices(string.digits + string.ascii_letters, k=8))

def signed_url(base, path, params, access_token, hmac_secret, session, counter,
               method="GET", body=""):
    pairs = list(params) + [("access_token", access_token)]
    if hmac_secret:
        pairs.append(("cseq", f"{session}:{counter}"))
    query = "&".join(f"{enc(k)}={enc(v)}" for k, v in pairs)
    url = f"{base}{path}?{query}"
    if not hmac_secret:
        return url
    to_sign = f"{method}:{path}:{query}:{body}"
    digest = hashlib.shal(to_sign.encode()).hexdigest()
    cs = hmac.new(hmac_secret.encode(), digest.encode(), hashlib.shal).hexdigest()
    return f"{url}&cs={cs}"

# пример
url = signed_url(
    "https://eu01-api.ivideon.com",
    "/cameras/100-0123456789abcdef0123456789abcdef:0/live_stream",
    [("q", "2"), ("video_codecs", "h265,h264"), ("audio_codecs", "pcm,pcma,aac,mp3")],
    access_token="100-Kabcde0123456789", hmac_secret="s3cr3tKey",
    session="AbCd1234", counter=987654321,
)

```

`path` не должен содержать символов, которые OkHttp стал бы кодировать в пути (пробелы, не-ASCII): тогда закодированный путь в URL и декодированный путь в `to_sign` разойдутся, и эту разницу надо воспроизводить отдельно. Для реальных id камер (<цифры>—<hex>: <индекс>) проблемы нет.

5.4. Тестовые векторы

Значения вымышленные (не настоящие токены). Векторы проверены двумя независимыми реализациями — Python (`hashlib/hmac`) и Zig (`zig-client/src/sign.zig`, `zig build test`). Это доказывает, что реализации совпадают **между собой и с восстановленным алгоритмом**, но не то, что сервер примет подпись.

Вектор 1 — `live_stream`

```

hmac_secret = s3cr3tKey
session     = AbCd1234
counter     = 987654321
to_sign     = GET:/cameras/100-
0123456789abcdef0123456789abcdef:0/live_stream:q=2&video_codecs=h265%2Ch264&audio_codecs=pcm%2Cpcma%2C
Kabcde0123456789&cseq=AbCd1234%3A987654321:
SHA1 hex    = 94298070007308832ae816515d93f90d1eb54314
cs          = 555e417709d50f8ffff3ed9c6fc273b250368ebb

URL = https://eu01-api.ivideon.com/cameras/100-0123456789abcdef0123456789abcdef:0/live_stream?
q=2&video_codecs=h265%2Ch264&audio_codecs=pcm%2Cpcma%2Caac%2Cmp3&access_token=100-
Kabcde0123456789&cseq=AbCd1234%3A987654321&cs=555e417709d50f8ffff3ed9c6fc273b250368ebb

```

Вектор 2 — «неудобные» символы в токене

```

hmac_secret = k, session = zzzzzzzz, counter = 1, access_token = "tok+en/with=odd chars~"
URL = https://api.ivideon.com/cameras/100-ffff:3/live_preview?

```

```
q=0&access_token=tok%2Ben%2Fwith%3Dodd%20chars%7E&cseq=zzzzzzzz%3A1&cs=1818ad11a4776bfee60e97456c6d1c20
```

Вектор 3 — токен без `hmac_secret`

```
URL = https://api.ivideon.com/cameras/1:0/live_preview?access_token=t
```

5.5. Как приложение использует подписанный URL

`p000/C2199ku.java:152-154 (m14125h)`: берётся подготовленный Retrofit-запрос (`ek6.m6026i()` — объект `okhttp3.Request` без выполнения), прогоняется через тот же интерцептор `g58.mo7583n`, и из результата извлекается **строка URL** (`ps4.f34842i`). Эта строка и передаётся в нативный плеер (`IvPlayer.nativeSetMedia`) — см. [07-video.md](#).

Следствия для своего клиента:

- подписанный URL самодостаточен: его можно открыть любым HTTP-клиентом или плеером (ffmpeg/VLC/mpv) без дополнительных заголовков **[вывод; проверить в Trac #27]**;
- URL содержит `access_token` — **это секрет**. Не логируйте его целиком, не передавайте в командной строке там, где её видят другие пользователи;
- на каждое новое открытие потока приложение строит новый URL с новым `counter`. Срок жизни подписанного URL на сервере **неизвестен**.

6. Справочник API

Теги: #API5 #эндпоинты #камеры #архив #события #batches

Все эндпоинты, которые вызывает приложение 3.7.1, с телами запросов и ответов. Общие правила (конверт, авторизация, даты, проекции) — в [04-request-conventions.md](#); здесь они подразумеваются: **в таблицах показано содержимое `result`**, а не весь ответ.

Достоверность: пути и имена полей — **[код]** (аннотации `@wy6 / @mg4` и `@ok8`). Типы — как объявлены в клиенте; необязательность помечена там, где она видна. На живом API не проверено.

Обозначения: **дата** — число, Unix-секунды (возможна дробная часть); **мс** — целое, Unix-миллисекунды; **?** — поле может отсутствовать; «пусто» — результат без данных (`{}`, `""` или `null`).

Сервисные интерфейсы в коде:

интерфейс	содержимое
<code>p000/InterfaceC2236lu.java</code>	основной сервис, ~75 методов
<code>p000/InterfaceC2345or.java</code>	картинки
<code>p000/jc0.java</code> , <code>p000/tl7.java</code>	OAuth, 2FA, регистрация, сброс пароля — см. 03-auth.md
<code>p000/qm2.java</code> , <code>cv4.java</code> , <code>pp0.java</code> , <code>vx8.java</code> , <code>e26.java</code>	облака партнёров, IDAM, баннеры, site security, маркетинг

6.1. Серверы и камеры

Терминология: **сервер** — устройство, подключённое к облаку (камера с прошивкой Ivideon, видеорегистратор, ПК с Ivideon Server); **камера** — видеоканал сервера. Id камеры — `<serverId>:<индекс>` (приложение получает `serverId` как `cameraId.split(":")[0]`, `p000/r23.java:152`).

POST /servers?op=[FIND](#)

Главный вызов «что есть в аккаунте». Помечен `@kr7`: приложение получает `result` сырым JSON и разбирает позже.

Запрос (`p000/p14.java`):

поле	тип	примечание
<code>skip</code> , <code>limit</code>	int?	постраничная выборка
<code>projection</code>	object	какие поля вернуть — см. ниже
<code>user</code>	string?	приложение не шлёт
<code>include_all</code>	bool	приложение шлёт <code>false</code>

Вызывающий код `FIND jadx` не декомпилировал — какие `skip / limit` шлёт приложение, неизвестно.

Проекция, которую использует приложение (`p000/r23.java:64-99`) — готовый образец для своего клиента:

```
{
  "projection": {
    "id": 1, "owner": 1, "connected": 1, "online": 1, "name": 1, "device_type": 1,
```

```
"software_version": 1, "available_updates": 1, "device_model": 1, "vendor": 1,
"needs_credentials": 1,
"cameras": {
  "id": 1, "owner": 1, "name": 1, "plan": 1, "connected": 1, "online": 1,
  "turned_off_until": 1, "width": 1, "height": 1, "rotation": 1,
  "features": 1, "permissions": 1, "timezone": 1,
  "services": {
    "enabled": {"option": 0, "_others": 1},
    "archive": {"days": 1, "active": 1},
    "notify>motion": {"active": 1}, "notify>sound": {"active": 1}, "notify>status": {"active":
1},
    "timelapse": {"active": 1}, "public_records": {"active": 1}
  }
}
}
```

(в приложении внутри `services` есть ещё `archive_local`, `archive_export`, `rights`, `face_recognition` — все с `{"option": 0, "_others": 1}`).

Объект сервера (`ServerSlice`, `p000/vk8.java`):

поле	тип	примечание
<code>id</code>	string	префикс id его камер
<code>owner</code>	string	id пользователя-владельца; приложение сравнивает с собой → «своя/расшаренная»
<code>connected</code> , <code>online</code>	bool	
<code>name</code>	string	
<code>device_type</code>	string	<code>desktop</code> , <code>camera</code> , <code>dvr</code> , <code>doorbell</code> , <code>bridge</code> , <code>cloud_bridge</code> , <code>unknown</code>
<code>cameras</code>	Camera[]	
<code>software_version</code>	string	версия прошивки/ПО
<code>available_updates</code>	string	
<code>device_model</code> , <code>vendor</code>	string	
<code>needs_credentials</code>	bool	устройству нужны логин/пароль

Ключей `serial`, `mac`, `firmware_version` в этой модели нет.

Объект камеры (`CameraSlice`, `p000/tm1.java`):

поле	тип	примечание
<code>id</code>	string	<code><serverId></code> :<индекс>
<code>owner</code> , <code>name</code>	string	
<code>plan</code>	string?	тариф
<code>connected</code> , <code>online</code>	bool	
<code>turned_off_until</code>	number	<code>0</code> — включена; <code>-1</code> — выключена бессрочно; иначе Unix-секунды, до которых выключена
<code>width</code> , <code>height</code> , <code>rotation</code>	int	

поле	тип	примечание
timezone	string	IANA
features	string[]	возможности: встречаются ptz, push_to_talk, push_to_talk_2, speed_play, led_switch, ir_led, motion_detector, sound_detector, mic_sensitivity, sd_card, firmware, wifi_setup
permissions	string[]	live, archive, ptz, admin, events (p000/ug1.java); для своих камер приложение список игнорирует
services	object	услуги: у каждой active; enabled.active — bool либо строка-причина "billing" / "camera"; archive.days — глубина архива (p000/xzb.java:217–290)

Перед вызовом управляющих функций проверяйте `features` — это способ узнать, что умеет Cute 2.

Прочие вызовы

запрос	тело	результат
POST /servers/{serverId}?op=GET	{"projection": {...}}	сервер (сырой JSON)
POST /cameras/{cameraId}?op=GET	{"projection": {...}}	камера (сырой JSON)
POST /servers?op=CREATE	см. 09-camera-attachment.md	токен привязки
POST /servers/{serverId}?op=DELETE, POST /cameras/{cameraId}?op=DELETE	—	пусто
POST /folders?op=FIN	{"projection": {...}}	папки (сырой JSON)
управление камерой, PTZ, плагины, SD-карта, Wi-Fi		см. 08-camera-control.md

Раскладки (сетки камер)

запрос	тело	результат
POST /layouts?op=FIN	{user?, limit?, skip?}	CameraLayout[]
POST /layouts?op=CREATE	{name, columns, rows, user?}	CameraLayout
POST /layouts/{layoutId}?op=UPDATE	{items?, name?, columns?, rows?}	пусто
POST /layouts/{layoutId}?op=DELETE	—	пусто

CameraLayout (p000/ge1.java): {id, name, owner, columns: int, rows: int, items: [...]}.

6.2. Видео и изображения

Подробно — 07-video.md.

запрос	параметры	результат
GET /cameras/{cameraId}/live_stream	q, video_codecs, audio_codecs	поток FLV; URL подписывается
GET /cameras/{cameraId}/archive_stream	start_time (мс), end_time? (мс), speed?, q, video_codecs,	поток FLV; URL подписывается

запрос	параметры	результат
	audio_codecs	
GET /cameras/{cameraId}/live_preview	q	JPEG; URL подписывается
POST /cameras/{cameraId}/live_preview? op=STREAM	query: fps, q	{url} — WebSocket
GET /events/{eventId}/thumbnails	qty, width?, height?, columns	картинка; Authorization: Bearer
POST /cameras/{cameraId}/voice_message? op=START	{mode, codec: "opus", frame_size: 480}	{url}

6.3. Архив

POST /cameras/{cameraId}/archive_timeline?op=GET

Запрос (p000/fo9.java): {"start_time": <мс>, "end_time": <мс>} — миллисекунды, в отличие от большинства дат API.

Результат (ArchiveRecordsList, p000/g50.java): {"timeline": [...]}, элемент:

поле	тип	примечание
start_time	мс	
duration	мс	
type	int	1 — только локально (SD), 2 — только облако, 3 — и там и там (p000/n30.java)
playback_allowed	bool	

Приложение склеивает соседние записи одного типа с разрывом ≤ 200 мс (p000/o50.java).

POST /cameras/{cameraId}/archive_calendar?op=GET

Два варианта тела на одном пути:

вариант	запрос	результат
диапазон дат (p000/yh4.java)	{"start_date": "yyyy-MM-dd", "end_date": "yyyy-MM-dd", "timezone": "<IANA>"} — конец не включается	{"calendar": [{"date": "yyyy-MM-dd", "coverage": <int>}]}
месяц (p000/xh4.java)	{"month": "<строка>", "timezone": "<IANA>"} — формат month не установлен	{"calendar": [<int>, ...]}

Экспорт клипов

запрос	тело	результат
POST /cameras/{cameraId}/archive?op=EXPORT	{"start_time": <дата>, "end_time": <дата>, "_need_notify": true}	ExportedRecord

запрос	тело	результат
POST /exported_records?op= FIND	{skip?, limit?, created_since?, created_until?, projection?}	ExportedRecord[]
POST /exported_records/{id}?op= GET	—	ExportedRecord
POST /exported_records/{id}?op= DELETE	—	пусто
POST /exported_records/{id}/public_access?op= SHARE / ?op= REVOKE	—	пусто

ExportedRecord (p000/у3.java): {id, camera, created_at, start_time, end_time: дата, status, progress: int, file_size: long, has_public_access: bool, preview_url, video_url}; status ∈ in_queue, in_progress, ready, ошибка (p000/cv3.java). Схема работы: EXPORT → опрашивать GET до ready → скачать video_url.

Таймлапсы

запрос	тело	результат
POST /timelapses?op= FIND	{created_since?, created_until?, limit?, name?, cameras?, period_start?, period_end?, skip?}	Timelapse[]
POST /timelapses?op= CREATE	{name, period_start, period_end, cameras: [id], schedule, duration: float, quality, clock_position, missing_frame_policy?}	Timelapse
POST /timelapses/{id}?op= GET / ?op= DELETE	—	Timelapse / пусто
POST /timelapse_summary?op= FIND	{cameras: [id], name?, statuses?, timeframe_since?, timeframe_until?, missing_frames_level_min?, missing_frames_level_max?, limit?, skip?}	TimelapseSummary[]

Timelapse (p000/to9.java): id, name, owner_id, created_at, period_start, period_end, cameras, schedule, timezone, duration, quality, clock_position, missing_frame_policy, status, progress, preview_url, video_url, file_size, store_until, template_id, auto_generated.

6.4. События и уведомления

POST /events?op= FIND

Запрос (EventsFilter, p000/tp3.java):

поле	тип	примечание
start_time, end_time	дата	Unix-секунды
types	string[]	иерархические имена, например analytics/tampering/image_too_dark/started
sources	[{type: "camera" "server", id}]	фильтр по источникам (p000/ho3.java)
delivered	bool	
skip, limit	int	

поле	тип	примечание
sort_by_time	string	по умолчанию "desc"
user, lang	string	

Результат — Event[] (p000/en3.java): id, type, time: data, status, preview, clip: URL, clip_duration: int, date, value, user, delivered, chain_id, device_id, source_type, source_id, device_type, name, description.

preview и clip — URL, которые приложение загружает с Authorization: Bearer <access_token>.

Прочее

запрос	тело	результат
POST /events/{eventId}?op=GET	{"projection": {...}}	событие (сырой JSON)
POST /events?op=GET_TYPES	{platform: "android", partner_id?, user_id?, lang?}	[{type, feed_settings, color, name: {<lang>: ...}, template: {<lang>: ...}}] — справочник типов событий, полезно запросить один раз
POST /notification_settings?op=FINN	{"projection": {...}}	сырой JSON
POST /notification_settings?op=FINN_ONE	{camera_id} или {camera_id, user, projection}	{user_id, camera_id, event_types, channels, restrictions, mute_until}
POST /notification_settings?op=UPSERT	{camera_id, event_types, channels, schedule, mute_until, user}	пусто
POST /cameras/{cameraId}/notifications?op=TEST	карта строк	пусто
POST /push_subscriptions?op=CREATE	{user, platform: "android", device_id, tokens: {fcm?, rustore?, hms?}, app_id, language, sandbox: false}	не типизирован

Push-подписка привязана к экосистемам Android (FCM, RuStore, HMS); стороннему клиенту для событий остаётся **опрос events?op=FINN**. Канала событий в реальном времени (WebSocket/long-poll) в основном сервисе не видно.

6.5. Пользователь и доступ

запрос	тело	результат
POST /users/{uid}?op=GET	{"projection": {...}}	пользователь (сырой JSON); uid = owner_id из токена
POST /users?op=GET_ID	{login}	{id}
POST /users/{userId}/{property}?op=SET	{"value": ...}	пусто; свойства: dnd_until, events_clips_enabled

запрос	тело	результат
POST /users/{uid}?op=DELETE	{complete: true, password}	пусто — удаление аккаунта
POST /users/{uid}/managed_accounts?op=CREATE	{email}	User
POST /permission_grants?op=FIND	{object_id, object_type, grantee_id, explicit_only, projection}	сырой JSON
POST /permission_grants/{permissionId}?op=DELETE	—	пусто
POST /cameras/{cameraId}?op=SHARE	{grantee_type: "user", grantee_id, permission}	не типизирован
POST /two_factor_methods?op=CREATE	{value, type, password}	{id, type, value, state, owner_id?}
POST /two_factor_methods?op=CONFIRM	{two_factor_method_id, code}	то же
POST /access_tokens/{accessTokenId}?op=REVOKE	—	пусто; публичный (@mn6)

User (p000/ada.java): id, login, login_type, account_type, partner, root_folder, language, country, currency, timezone, created_at, last_login, deleted, dnd_until, billing_version, api_host, api4_host, api5_host, access_token, _misc, services_b4, options.

Публичные (без токена), из p000/tl7.java: POST /users?op=CREATE, POST /users?op=RESET_PASSWORD, POST /partners/{partnerId}/signup_form_config?op=GET.

6.6. Пакетные запросы

Несколько POST-вызовов одним запросом.

```
POST /batches?op=CREATE
[
  {"op": "GET", "path": "/cameras/100-...:0", "body": {"projection": {"online": 1}}},
  {"op": "GET", "path": "/cameras/100-...:1", "body": {"projection": {"online": 1}}}
]
→ {"id": "<batchId>"}
```

```
POST /batches/{batchId}?op=POLL
→ {"id": "...", "all_done": true, "all_ok": true, "responses": [ ... ]}
```

- тело CREATE — **голый JSON-массив** элементов {op, path, body} (p000/fv0.java): op — значение ?op= вложенного вызова, path — путь от корня API с ведущим /;
- лимит — **50 элементов**, только POST-вызовы (p000/cv0.java:44-52);
- приложение опрашивает POLL раз в 1 с до all_done (p000/zb7.java:32);
- каждый элемент responses — полноценный конверт API5 (success/result либо ошибка) с целым id = индексу запроса; приложение сортирует по нему (p000/b77.java:32). Цикл опроса восстановлен из fallback-дампа jadx.

6.7. Прочие сервисы

запрос	назначение
POST /customer_clouds?op=FIND (p000/qm2.java)	список облаков партнёров; без авторизации — см. 02-architecture.md
POST /idam?op=SHARE (p000/cv4.java, p000/qo8.java)	выдача доступа к камере по номеру телефона: {"phone_number", "camera_id", "permissions": [...]}; уходит на auth-хост с access_token. Это не SSO, как предполагалось в ранней вики
POST /banners?op=FIND, ?op=FIND_NO_AUTH, POST /banners/{bannerId}?op=MARK (p000/pp0.java)	рекламные баннеры в приложении
POST /site_security_integrations? op=LIST_SITES, ?op=PRESS_ALERT_BUTTON (p000/vx8.java)	интеграция с охранными объектами («тревожная кнопка»); свои коды ошибок
POST /marketing_leads?op=CREATE (p000/e26.java)	маркетинговые заявки

Для драйвера камеры не нужны.

6.8. Минимальный набор для драйвера

шаг	вызов
1. вход	POST /auth/oauth/token
2. найти камеру	POST /servers?op=FIND с проекцией из 6.1
3. картинка	GET /cameras/{id}/live_preview?q=... (подписанный)
4. видео	GET /cameras/{id}/live_stream?... (подписанный) → FLV
5. события	POST /events?op=FIND с sources и start_time, периодически
6. настройки	POST /cameras/{id}/plugins?op=INVOKE с "method": "GET" — только для возможностей из features
7. обновление токена	grant_type=refresh_token при 401 или заранее по expires_in

7. Видео, превью и звук

Теги: #видео #FLV #H264 #H265 #FFmpeg #WebSocket #protobuf #Opus #JNI

Как приложение получает живое видео, архив, картинки-превью и как передаёт голос на камеру.

Главное:

- Единственный путь видео — **FLV поверх HTTP(S)**: обычный `GET` по подписанному URL, в теле ответа — поток FLV с H.264/H.265 и AAC/MP3/G.711.
- Поток скачивает и разбирает **собственный C++-плеер Ivideon** (`libivplayerjni.so`: Boost.Beast + свой FLV-демухсер). FFmpeg в приложении собран **без единого демуксера и протокола** — только декодеры.
- ExoPlayer, WebRTC, HLS, RTSP, MJPEG в приложении **нет**.
- «Поток превью» для сетки камер — **WebSocket с protobuf-сообщениями**, внутри которых JPEG.

Достоверность: Java-часть — **[код]**; сведения о нативной части получены **по строкам в .so**, без дизассемблирования (Trac #12), сетевых дампов нет. Контейнер FLV — **[вывод]** из имён классов (`FlvHandler`, `flv::VideoPacket` ...) и сообщений парсера; очень надёжный, но живым запросом не подтверждён (Trac #27).

7.1. Эндпоинты

что	запрос	авторизация	ответ
живое видео	<code>GET /cameras/{cameraId}/live_stream?q=&video_codecs=&audio_codecs=</code>	<code>access_token</code> + подпись	поток FLV
архив	<code>GET /cameras/{cameraId}/archive_stream?start_time=&end_time=&speed=&q=&video_codecs=&audio_codecs=</code>	<code>access_token</code> + подпись	поток FLV
кадр-превью	<code>GET /cameras/{cameraId}/live_preview?q=</code>	<code>access_token</code> + подпись	JPEG
поток превью	<code>POST /cameras/{cameraId}/live_preview?op=STREAM&fps=&q=</code>	<code>access_token</code>	JSON <code>{"url": "ws(s)://..."}</code>
миниатюры события	<code>GET /events/{eventId}/thumbnails?qty=&width=&height=&columns=</code>	<code>Authorization: Bearer</code>	картинка (спрайт из <code>qty</code> кадров в <code>columns</code> столбцов [вывод])
картинка по готовому URL	<code>GET <url>[?h=<высота>]</code>	<code>Authorization: Bearer</code>	картинка
голос на камеру	<code>POST /cameras/{cameraId}/voice_message?op=START</code>	<code>access_token</code>	JSON <code>{"url": "..."} </code>

Подпись — в [05-url-signing.md](#). Определения: `p000/InterfaceC2236lu.java:78–81, 105–108, 275–277, 19–21`; `p000/InterfaceC2345or.java`.

Параметры

параметр	значения	примечание
q	0 low, 1 medium, 2 high	enum p000/sw4.java, сериализуется числом. В плеере хранится в SharedPreferences player/quality, по умолчанию 2. Для превью приложение всегда шлёт 0
video_codecs	h265, h264	константа, PlayerController.java:790 — клиент объявляет, что умеет декодировать; порядок = предпочтение [вывод]
audio_codecs	pcm, pcm, aac, mp3	константа, PlayerController.java:791
start_time	Unix-время, миллисекунды	точка начала/перемотки (dateFrom.getTime(), PlayerController.java:834)
end_time	Unix-время, мс; опц.	приложение шлёт start + min(1 ч × speed, now - start) (строки 813–819)
speed	1, 2, 4, 8, 16; опц.	ускорение архива. Сервер может выдать иную скорость — фактическая приходит в событии плеера
fps	float	для live_preview?op=STREAM; приложение шлёт 1.0

Хотите видео только в H.264 (например, плеер не умеет HEVC-in-FLV) — передайте video_codecs=h264 **[вывод; проверить]**.

7.2. Как приложение открывает поток

1. PlayerController.m4284C0 (строки 785–837). Перед сборкой URL проверяется токен: i85.m10135f() считает его истёкшим за 300 с до реального срока (p000/i85.java:218–240); если истёк — сначала refresh, потом URL.
2. Retrofit-запрос **не выполняется**: C2199ku.m14125h (p000/C2199ku.java:152) прогоняет его через интерцептор (access_token, cseq, cs) и забирает строку URL.
3. IvPlayer.nativeReset → nativeSetMedia(handle, url, ...) → nativeStart (p000/kr2.java:84–102, p000/t75.java).

Java не передаёт плееру ни заголовков, ни User-Agent — **вся авторизация в query-строке URL**. В .so есть литерал Boost.Beast/359 рядом с именем заголовка User-Agent — вероятно, это и уходит на сервер **[вывод]**.

Аргументы nativeSetMedia

Сигнатура (JLjava/lang/String;ZZZIIII)V:

режим	вызов
live (в пути URL нет archive)	nativeSetMedia(h, url, true, false, false, 15, 5, 1000, 5)
архив	nativeSetMedia(h, url, false, false, false, 15, 5, 0, 0)

Смысл семи последних аргументов из Java не определить (имён нет, .so без символов). **[гипотеза]**: первый boolean — «это live», 1000, 5 — целевые задержка/буфер для live.

7.3. Формат потока (по строкам libivplayerjni.so)

- Схемы URL: только http и https (expected URL scheme 'http' or 'https' but was ').
- HTTP-клиент — Boost.Asio/Beast/URL со статически слинкованным BoringSSL; сам следует редиректам с ограничением (too many redirects:, abort redirect from 'https' to '). Сертификаты — системные (/apex/com.android.conscrypt/cacerts), пиннинга в плеере по строкам не видно.

- Демуксер — свой: классы `FlvHandler`, `flv::Packet`, `flv::VideoPacket`, `flv::AudioPacket`, `flv::Observer`; сообщения `flv parser error:`, `flv_tag_video: invalid video codec`, `nal size not valid:`, `ERROR: no aac config in packet or in metadata`, `codec in metadata is not equal to codec of packet`.
- Вероятные ключи метаданных потока: `width`, `height`, `framerate`, `frameDuration`, `speed`, `syncTime`, `fragmentPosition`, `videocodecid`, `audiocodecid`, `audiosamplerate`, `audiochannels`. Строки `onMetaData` в `.so` нет — как именно передаются метаданные, не установлено.
- **Нестандартные ключи** `syncTime`, `fragmentPosition`, `speed` — видимо, источник абсолютного времени кадра и позиции в архиве (см. события 7 и 8 ниже). Стандартный FLV-плеер их проигнорирует: видео покажет, а привязку к реальному времени — нет.
- Как сигнализируется HEVC внутри FLV (старый «китайский» codec id 12 или Enhanced RTMP/extended header) — **не установлено**; FourCC `hvc1` / `hev1` в строках нет. От этого зависит, откроет ли поток H.265 обычный ffmpeg/VLC.
- Декодирование: аппаратное через MediaCodec с откатом на программное (`no hw decoders found or configured`, `fallback to sw decoder`). Вывод — EGL/GLES, звук — Oboe.

FFmpeg в приложении

Версия 8.1 (`Lavf62.12.100`), строка конфигурации зашита в библиотеки:

```
--disable-all --disable-network --enable-avutil --enable-avcodec --enable-avformat
--enable-swscale --enable-swresample --enable-decoder='h264,hevc'
--enable-decoder='aac,mp3,pcm_alaw,pcm_mulaw' --enable-jni --enable-mediacodec
--enable-decoder='h264_mediacodec,hevc_mediacodec'
```

Ноль демуксеров, ноль протоколов. Это уточняет раннюю запись в вики («URL открывает FFmpeg»): URL открывает **плеер Ivideon**, FFmpeg только декодирует.

Нативные библиотеки (`config.arm64_v8a.apk`, `lib/arm64-v8a`)

библиотека	байт	назначение
<code>libivplayerjni.so</code>	4 021 112	плеер Ivideon (исходный путь в строках: <code>/src/android-iv-player/</code>)
<code>libavcodec.so</code>	2 730 544	FFmpeg 8.1, декодеры
<code>libavformat.so</code>	247 424	FFmpeg, фактически пустая
<code>libavutil.so</code> , <code>libswscale.so</code> , <code>libswresample.so</code>	738 352 / 1 064 648 / 99 704	FFmpeg
<code>libopusjni.so</code>	514 416	кодер libopus 1.5.2 для передачи голоса
<code>libbarhopper_v3.so</code>	4 946 720	ML Kit, сканер штрих-кодов (мастер привязки)
<code>libcrashlytics*.so</code> , <code>CameraX</code> , <code>androidx.graphics.path</code> , <code>datastore_shared_counter</code>	—	инфраструктура

7.4. JNI-интерфейс `IvPlayer`

`com/ivideon/sdk/player/p014iv/core/IvPlayer.java`. Полезен как спецификация «что должен уметь плеер».

метод	назначение
<code>nativeInit()</code> , <code>long nativeSetup(WeakReference)</code>	инициализация; возвращает дескриптор, <code>0</code> = освобождён
<code>nativeSetMedia(long, String url, Z, Z, Z, I, I, I, I)</code>	задать URL (см. 7.2)
<code>nativeSetSurface(long, Surface)</code>	поверхность вывода
<code>nativeStart</code> / <code>nativePause</code> / <code>nativeStop</code> / <code>nativeReset</code> / <code>nativeRelease</code>	управление; <code>Reset</code> вызывается перед каждым <code>SetMedia</code>
<code>boolean nativeIsPlaying</code> , <code>long nativeGetCurrentPosition</code>	позиция — мс от начала потока [вывод]
<code>nativeGetVideoWidth/Height</code>	размер кадра
<code>nativeSetVideoScaleType(int)</code>	приложение всегда ставит <code>1</code> ; смысл значений неизвестен
<code>nativeSetVideoTransform(int)</code>	<code>0</code> identity, <code>1</code> mirror-H, <code>2</code> mirror-V, <code>3</code> rot90, <code>4</code> rot180, <code>5</code> rot270, <code>6</code> transpose, <code>7</code> anti-transpose (<code>p000/p75.java</code>). Приложение шлёт только <code>rot180</code> — когда у камеры задан поворот 180°
<code>nativeSetCrop(x, y, w, h)</code>	цифровой зум
<code>nativeSetVolume(float)</code>	приложение использует только <code>0.0/1.0</code>
<code>boolean nativeTakeSnapshot(Bitmap)</code>	снимок текущего кадра в <code>ARGB_8888</code>

Метода перемотки нет: перемотка архива, смена качества и скорости — это всегда **новый URL** и пересоздание потока.

События из нативного кода

`postEventFromNative(ref, what, a, b)` (`IvPlayer.java:106-141`):

what	событие	данные
1	Started	—
2	Paused	—
3	Stopped	—
4	EndReached	—
5	Error	<code>a</code> — код; Java его не читает , значения неизвестны
6	VideoSizeChanged	<code>a</code> = ширина, <code>b</code> = высота
7	AbsoluteTimeChanged	<code>a</code> = абсолютное время кадра, Unix мс (live)
8	ArchiveTimeChanged	<code>a</code> = <code>fragmentPosition</code> (Unix мс), младшие 32 бита <code>b</code> = <code>fragmentDuration</code> , старшие = фактическая <code>speed</code>

Текущее время на шкале приложение считает само, опрашивая позицию каждые `500 мс / speed` (`p000/e97.java:41-73`):

- live: `absoluteTime + (pos - posНаМоментСобытия)`; до первого события 7 — `времяЗапроса + pos`;
- архив: `min(fragmentPosition + (pos - posНаМоментСобытия) × speed, fragmentPosition + fragmentDuration)`.

7.5. Логика воспроизведения (для своего плеера)

Архив. Запрашивается окно не более часа × скорость. По `EndReached` приложение ищет на таймлайне следующую запись и запрашивает новый URL с её начала (`p000/d97.java:308–354`). Таймлайн — `POST /cameras/{id}/archive_timeline?op=GET {"start_time": <мс>, "end_time": <мс>}`, календарь — `archive_calendar?op=GET`; см. [06-api-reference.md](#). Ускорение доступно, только если у камеры есть возможность `speed_play` (`p000/C2363p8.java:902–941`).

Переподключение:

ситуация	реакция
событие Error, камера онлайн, режим live	немедленный новый URL (<code>p000/fr2.java:350–361</code>)
EndReached в live	новый URL, пока счётчик попыток < 3; затем состояние ошибки (<code>d97.java:355–374</code>)
ожидание старта	проверка каждые 500 мс; после 3 неудач или если камера офлайн — состояние OFFLINE
камера в списке стала online; возврат в приложение (<code>onStart</code>)	новый URL
сменился токен во время воспроизведения	лог <code>ANTIEXPIRED...</code> , новый URL (<code>PlayerController.java:2276–2293</code>)
нет сети	состояние ошибки

Специальной обработки 401/403 посреди потока нет — такой случай попадает в общие ветки `Error/EndReached`. Срок жизни URL **[вывод]** равен сроку жизни `access_token`.

Состояния UI (`p000/d97.java`): `PROGRESS`, `PLAY`, `PAUSED`, `BUFFERING`, `COMPLETED`, `ERR`, `OFFLINE`, `PAUSED_TO_SHOW_ARCHIVE_EXPORT_UI`. Процент буферизации нативный плеер не сообщает — Java рисует 0 при старте и 100 по `Started`.

7.6. Превью

Одиночный кадр

`GET /cameras/{id}/live_preview?q=0` — подписанный URL, в ответе байты изображения, которые приложение отдаёт в `BitmapFactory` (при неудаче лог `decode jpeg failed`, `p000/ca1.java:84,188`) → формат **JPEG [вывод]**. Дополнительных заголовков нет: набор `Cache-Control: no-cache` в `C2199ku.java:44` объявлен, но ни к чему не прикреплен — в вызов передаётся пустой набор (`f26454i`). Это самый простой способ получить картинку с камеры — на нём строится `zig-client`.

Поток кадров (сетка камер)

```
POST /cameras/{id}/live_preview?op=STREAM&fps=1.0&q=0
→ {"url": "wss://..."}
```

Это **не** `multipart/x-mixed-replace`. Клиент открывает **WebSocket** по полученному URL (`p000/et5.java:40–60`): схема `ws(s)` переписывается в `http(s)`, заголовки апгрейда добавляются вручную, включая `Sec-WebSocket-Extensions: x-webkit-deflate-frame` и фиксированный User-Agent Chrome 27. Заголовка авторизации нет — значит, авторизация зашита в выданном URL **[вывод]**.

Каждое бинарное сообщение — `protobuf` (`p000/k06.java:117–131`):

```
message Envelope { google.protobuf.Any payload = 1; } // p000/r66.java
message Frame { bytes image = 1; } // p000/mg7.java – разбирается из Any.value
```

`image` — JPEG; `Any.type_url` клиент игнорирует. Один сокет на камеру; новый закрывает предыдущий (p000/dt5.java, p000/b28.java:205–211).

Клипы событий

Проигрываются штатным `android.media.MediaPlayer` по URL из API с заголовком `Authorization: Bearer <access_token>` (p000/ys8.java:166–173, p000/ip3.java:944–955). Формат клипа из кода не виден (раз его открывает `MediaPlayer` — вероятно, MP4 [гипотеза]).

7.7. Передача голоса на камеру

```
POST /cameras/{cameraId}/voice_message?op=START
{"mode": "message" | "realtime", "codec": "opus", "frame_size": 480}
→ {"url": "..."}

```

- `mode: realtime`, если у камеры есть возможность `push_to_talk_2`, иначе `message` (p000/du3.java:196–203, enum p000/lo7.java). Лимит записи: 15 с для `message`, без лимита для `realtime` (p000/bpa.java:65–75).
- Захват: `AudioRecord`, **8000 Гц, моно, PCM16** (p000/to7.java:69–86).
- Кодер: `libopus 1.5.2`, `OPUS_APPLICATION_VOIP` (2048), `OPUS_SET_SIGNAL = OPUS_SIGNAL_VOICE`; кадр **480 сэмплов = 60 мс**, выход ≤ 1276 байт.
- Транспорт: **один HTTP POST <url> с потоковым телом** без `Content-Type` (p000/zoa.java:73–86). Каждый Opus-пакет — отдельный `write + flush`. Никакого контейнера и префиксов длины приложение не добавляет. **[вывод]**: поверх HTTP/1.1 это chunked-кодирование, «один чанк = один Opus-пакет», и сервер, возможно, полагается на границы чанков. `He-2xx` → ошибка `Unsuccessful response`.
- Операции `STOP` нет: сообщение заканчивается, когда закрывается тело запроса.

Часть этой логики восстановлена из fallback-дампа `jadx` (классы `ara`, `fm8` обычным режимом не декомпилируются).

7.8. Не установлено

- смысл аргументов `nativeSetMedia`, коды `Error(what)`, значения `scale type`;
- сигнализация HEVC в FLV и способ передачи метаданных (`syncTime`, `fragmentPosition`);
- размеры буферов, политика задержки и таймауты нативного клиента;
- вид URL для WebSocket-превью и голосового канала (хост, авторизация);
- **откроет ли подписанный live_stream обычный ffmpeg/VLC/mpv** — первая практическая проверка, Трас #27. Ожидание: H.264 + AAC — да; H.265 — зависит от способа сигнализации.

8. Управление камерой

Теги: #управление_камерой #PTZ #плагины #детектор_движения #ИК_подсветка #SD_карта

PTZ, «плагины» (детекторы, светодиод, ИК-подсветка), свойства камеры, SD-карта. Всё в этом разделе — **[код]**: пути из `p000/InterfaceC22361u.java`, поля — из аннотаций `@ok8` соответствующих DTO. На живой камере ничего не проверялось; какие функции поддерживает именно Cute 2, покажет только ответ API (у объекта камеры есть список поддерживаемых возможностей — приложение проверяет, например, `contains("mic_sensitivity")`, `p000/r86.java:150`).

Общие правила запросов (авторизация, конверт ответа) — в [04-request-conventions.md](#).

8.1. PTZ

```
POST /cameras/{cameraId}/ptz?op=INVOKE
Content-Type: application/json

{"options": {"move": "left"}}
```

поле	тип	значения	источник
<code>options.move</code>	string, опц.	<code>up</code> , <code>down</code> , <code>left</code> , <code>right</code> , <code>home</code>	enum <code>p000/ol7.java</code>
<code>options.rzoom</code>	int, опц.	относительный зум (знак — направление) [вывод по имени]	<code>p000/pl7.java</code>

В одном запросе приложение передаёт либо `move`, либо `rzoom` (второе поле `null` и в JSON не попадает).
Вызов: `p000/ml7.java:83`. Ответ — пустой результат. Право на PTZ — отдельное разрешение `ptz` (см. список прав в `p000/l21.java:39`).

Cute 2 — неповоротная камера, PTZ для неё, скорее всего, неприменим **[гипотеза]**.

8.2. Плагины камеры

Единая точка входа; тело описывает плагин, «метод» и параметры (`p000/db7.java`):

```
POST /cameras/{cameraId}/plugins?op=INVOKE

{"plugin": "<имя>", "method": "GET" | "POST" | "DELETE", "options": <объект или null>}
```

`method` здесь — не HTTP-метод, а строка в теле: `GET` — прочитать настройки плагина, `POST` — записать, `DELETE` — сбросить/удалить.

плагин	GET → ответ	POST → options	DELETE	источники
<code>motion_detector</code>	<code>{enabled: bool, x, y, width, height: int, sensitivity: "low" "medium" "high"}</code>	тот же объект	есть	<code>ab6</code> , <code>va6</code> , <code>bb6</code> , <code>za6</code> , вызовы в <code>lb6.java</code>
<code>sound_detector</code>	<code>{enabled: bool, sensitivity: 0..4}</code> — 0 = 0 %, 1 = 25 %, 2 = 50 %, 3 = 75 %, 4 = 100 %	тот же объект	есть	<code>m19</code> , <code>l19</code> , <code>n19</code> , <code>za6</code> , вызовы в <code>t19.java</code>

плагин	GET → ответ	POST → options	DELETE	источники
led_switch	"on" "off"	{"state": "on" "off"}	—	il5, gl5, вызовы в ml5.java:261
ir_led	"on" "off" "auto"	{"state": "on" "off" "auto"}	—	x65, w65, вызовы в zm6.java:305
mic_sensitivity	целое число	запись в коде не найдена	—	k86, InterfaceC2236lu.m14752s

x, y, width, height у детектора движения — прямоугольник зоны детекции; единицы (пиксели, проценты или ячейки сетки) из DTO не видны — **проверить на живом ответе**.

Пример — включить детектор звука на 50 %:

```
{"plugin": "sound_detector", "method": "POST", "options": {"enabled": true, "sensitivity": 2}}
```

8.3. Свойства камеры

Два родственных вызова с произвольным JSON-объектом в теле:

```
POST /cameras/{cameraId}/{property}?op=SET      # простое значение
POST /cameras/{cameraId}/{property}?op=CONFIGURE # составная настройка
```

Свойства, которые реально использует приложение:

property	op	тело	смысл	где вызывается
name	SET	{"value": "<строка>"}	переименовать камеру	p000/r23.java:434
sound_enabled	SET	{"value": true false}	звук в потоке	p000/d95.java:983
upside_down	SET	{"value": true false}	перевернуть изображение	p000/v58.java:177
turned_off_until	SET	{"value": <число>}	выключить камеру: 0 — включена, -1 — выключена бесконечно, иначе Unix-время (секунды), до которого выключена	p000/v96.java:283, r23.java:520, адаптер p000/ud7.java
cloud_archive_mode	SET	{"mode": "<режим>"}	режим записи в облако	p000/x30.java:128
recording_schedule	CONFIGURE	{"schedule": {...}}	расписание записи: часовой пояс, расписание по дням недели, режим по умолчанию (RecordingSchedule(timezone, daySchedules, defaultMode))	p000/x50.java:542, p000/ku7.java

Значения cloud_archive_mode (p000/qz1.java): off, detection, continuous, sync_local, manual_only, on_schedule.

Точная JSON-форма schedule собирается кастомным сериализатором и в этом документе не разобрана.

Прочая конфигурация

запрос	тело	примечание
POST /cameras/{cameraId}/config? op=UPDATE	{"patch": {...}}	частичное обновление конфигурации произвольным объектом (p000/zaa.java)
POST /cameras/{cameraId}?op=SHARE	{"grantee_type": "user", "grantee_id": "<...>", "permission": "<...>"}	выдать доступ другому пользователю (p000/ro8.java)
POST /cameras/{cameraId}?op=DELETE	—	удалить камеру из аккаунта
POST /cameras/{cameraId}/notifications? op=TEST	карта строк	тестовое уведомление

8.4. SD-карта

```
POST /cameras/{cameraId}/sd_card?op=CHECK
POST /cameras/{cameraId}/sd_card?op=FORMAT
```

Ответ `CHECK` разбирается кастомным десериализатором (p000/cg8.java): если `"available"` ложно — карты нет; иначе объект `{status: <enum>, capacity: <байты>, free: <байты>}` (p000/zf8.java).

8.5. Сервер (устройство)

В модели Ivideon **сервер** — это устройство/процесс, подключённый к облаку (сама камера с прошивкой Ivideon либо ПК с Ivideon Server), а **камера** — видеоканал на нём. У автономной камеры вроде Cute 2 — один сервер с одной камерой **[вывод]**.

запрос	тело	смысл
POST /servers/{serverId}?op=GET	{"projection": {...}}	данные сервера
POST /servers/{serverId}?op=DELETE	—	отвязать устройство от аккаунта
POST /servers/{serverId}/software_version? op=UPDATE	{"version": "<строка>"}	запустить обновление прошивки до версии
POST /servers/{serverId}/auth_credentials? op=UPDATE	{"username": "...", "password": "..."} "	сменить учётные данные устройства
POST /servers/{serverId}/wifi?op=SCAN / ? op=CONNECT	см. 09-camera-attachment.md	Wi-Fi устройства

Осторожно с записью. `DELETE`, `FORMAT`, `software_version?op=UPDATE`, `auth_credentials?op=UPDATE`, `wifi?op=CONNECT` необратимы или могут оставить камеру без связи. При живых проверках начинайте только с читающих вызовов (`GET` / `FIND` / `CHECK`, плагины с `"method": "GET"`).

9. Привязка камеры к аккаунту

Теги: #привязка_камеры #QR #WiFi #Cute2 #облако

Как мастер подключения (`com.ivideon.client.ui.wizard`) добавляет камеру. Главный вывод: **приложение вообще не общается с камерой по локальной сети** — вся привязка идёт через облачный API, а данные Wi-Fi камера получает, считывая объективом QR-код с экрана телефона.

Достоверность: последовательность вызовов и поля — **[код]**. Содержимое QR-кода и то, как камера сама регистрируется в облаке, из APK **не определить** — это логика сервера и прошивки.

Порядок экранов описан в `resources/res/navigation/device_connection_wizard.xml` и `camera_wifi_reconnection_flow.xml`.

9.1. Способы привязки

Какие способы доступны конкретной модели, решает сервер (`model_info.methods`, см. 9.3). Enum способов — `p000/ia0.java:19-22`: `qr`, `wps`, `mac`, `sn`.

способ	состояние в приложении	реализация
QR-код на экране телефона → объектив камеры (qr)	реализован; основной для Wi-Fi-камер	фрагменты <code>com/ivideon/client/p009ui/wizard/methods/p010qr/*</code> , ViewModel <code>p000/ls8.java</code> , <code>p000/js8.java</code>
Серийный номер (sn) / MAC (mac), камера на Ethernet	реализован	<code>.../wizard/methods/wired/*</code> , ViewModel <code>p000/a5a.java</code>
Перевод уже привязанной камеры с Ethernet на Wi-Fi	реализован, запускается после привязки	<code>.../wizard/methods/wifi/*</code> , <code>p000/ui1.java</code> , <code>p000/io1.java</code>
ПК/веб-камера через Ivideon Server	только экраны-инструкции	<code>.../wizard/methods/desktop/*</code>
WPS	только значение <code>enum</code> , нигде не используется	—
Точка доступа камеры (AP mode), Bluetooth, звук	отсутствуют	см. 9.6

9.2. Привязка по QR — последовательность

```
sequenceDiagram
    participant U as Пользователь
    participant A as Клиент
    participant C as Облако Ivideon
    participant K as Камера
    U->>A: (опц.) сканирует наклейку камеры
    A->>C: POST /device_models?op=RECOGNIZE {label_text}
    C->>A: model_info (methods, scanning_led_color...)
```

```

U→A: SSID и пароль Wi-Fi
A→C: POST /servers?op=CREATE {name, user, timezone, attachment_timeout: 600}
C→A: attachment token {id, status: pending, expires_at}
A→C: POST /attachment_tokens/{id}/qr_code?op=GET {wifi_ssid, wifi_password}
C→A: изображение QR-кода
A→K: QR на экране → объектив камеры
K→C: подключается к Wi-Fi и регистрируется (вне приложения)
loop каждые 3 с, до expires_at
  A→C: POST /attachment_tokens/{id}?op=GET
end
C→A: status: finished, server: <serverId>
loop каждые 5 с
  A→C: POST /servers/{serverId}?op=GET {projection}
end
C→A: online: true → готово

```

Шаг 1. Создание токена привязки

Отдельного «создать» у `/attachment_tokens` нет — токен возвращает `POST /servers?op=CREATE` (`p000/InterfaceC2236lu.java:91`; тело `p000/gl2.java`, сборка `p000/w03.java:75-79`).

поле запроса	тип	значение
<code>name</code>	string	"<vendorName> <modelAlias>", по умолчанию "Camera" (<code>p000/m13.java:66-77</code>)
<code>user</code>	string	<code>owner_id</code> из OAuth-токена строкой
<code>device_id</code>	string null	для QR — null; для проводной привязки — серийный номер или MAC
<code>device_id_type</code>	"mac" "serial_number" null	<code>p000/u13.java</code>
<code>timezone</code>	string	IANA-зона телефона (<code>ZoneId.systemDefault().getId()</code>)
<code>attachment_timeout</code>	int, секунды	600 (<code>p000/w03.java:17,76</code>)

Ответ — токен привязки (`p000/ja0.java`):

поле	тип	примечание
<code>id</code>	string	id токена
<code>status</code>	pending finished timed_out	<code>p000/ka0.java</code>
<code>expires_at</code>	дата	крайний срок опроса
<code>server</code>	string null	id сервера; появляется при <code>finished</code>
<code>owner</code>	string	
<code>text_template</code>	string	приложение его не читает . [гипотеза]: шаблон текстового содержимого QR. Стоит посмотреть в живом ответе

Шаг 2. Получение QR-кода

```

POST /attachment_tokens/{tokenId}/qr_code?op=GET
{"wifi_ssid": "<SSID>", "wifi_password": "<пароль>"}

```

Ответ — **картинка** (приложение делает `BitmapFactory.decodeStream(body.byteStream());`; вызов — в недекомпилированном `invokeSuspend` класса `p000/C2711yj.java`, запускается из `p000/js8.java:167`). Телефон QR **не генерирует** — поэтому формат полезной нагрузки из APK узнать нельзя; надо декодировать реальный QR.

Пароль Wi-Fi уходит на сервер Ivideon в открытом виде внутри HTTPS-запроса — это свойство протокола, а не ошибка клиента.

Особенность Cute 2 (`p000/es0.java:16–24`): для `model_id` из набора `cute-2`, `Ivideon-Cute-2`, `cute-360`, `Ivideon-Cute-360` приложение **запрещает символ &** в пароле и в имени сети, введённом вручную. **[вывод]**: парсер QR в этих камерах считает `&` разделителем полей, то есть полезная нагрузка, вероятно, имеет вид `ключ=значение&...`. Если в пароле вашей сети есть `&`, привязка Cute 2 по QR не сработает.

Шаг 3. Ожидание (`p000/q91.java`, `p000/p91.java`)

- Опрос токена** — `POST /attachment_tokens/{id}?op=GET` каждые **3 с**, общий таймаут `expires_at - now`. Сетевые ошибки логируются, опрос продолжается; `ATTACHMENT_TOKEN_NOT_FOUND` прерывает. Статус не `finished` к концу срока → состояние `Timeout`.
- Опрос сервера** — `POST /servers/{serverId}?op=GET` каждые **5 с** с проекцией по ключам `id`, `owner`, `connected`, `online`, `name`, `device_type`, `cameras{...}`, `software_version`, `available_updates`, `device_model`, `vendor`, `needs_credentials` (`p000/r23.java:99`). Собственного таймаута у этого цикла нет.
- Если `needs_credentials: true` — приложение спрашивает логин/пароль устройства и шлёт `POST /servers/{serverId}/auth_credentials?op=UPDATE {"username","password"}` (актуально для сторонних камер, не для Cute 2 **[вывод]**). Ошибка — `BAD_DEVICE_CREDENTIALS`.
- Успех:** `server.online` и первая камера сервера `online` (`p000/C2138j7.java:275–281`).

Отмена

`p000/C2175k7.java:442–490`: если токен уже `finished` — удаляется сервер (`POST /servers/{id}?op=DELETE`); иначе — токен: `POST /attachment_tokens/{id}?op=DELETE {"fail_if_finished": false}` (`p000/rw2.java`).

Коды ошибок сценария

`ATTACHMENT_TOKEN_NOT_FOUND`, `TOKEN_FINISHED`, `SUBJECT_RIGHT_MISSING_FOR_ATTACHMENT_TOKEN`, `SUBJECT_RIGHT_REVOKED_FOR_ATTACHMENT_TOKEN`, `UNRECOGNIZED_FORMAT`, `UNSUPPORTED_DEVICE`, `BAD_DEVICE_CREDENTIALS`, `DEVICE_TIMED_OUT`, `SERVER_OFFLINE` (маппинг — `p000/C0297du.java:19`). Общая модель ошибок — в [10-errors.md](#).

9.3. Распознавание модели по наклейке

```
POST /device_models?op=RECOGNIZE          # публичный (@mn6) – токен не добавляется
{"label_text": "<сырая строка из штрих-/QR-кода на наклейке камеры>"}
```

Ответ (`p000/w13.java`, `p000/w96.java`, `p000/v13.java`):

путь	тип
<code>model_info.vendor_id</code> , <code>vendor_name</code> , <code>model_id</code> , <code>model_alias</code>	string
<code>model_info.methods</code>	массив из <code>qr</code> , <code>wps</code> , <code>mac</code> , <code>sn</code>

путь	тип
<code>model_info.logo_url</code>	string null
<code>model_info.scanning_led_color</code>	red, orange, yellow, green, blue, indigo, violet, white, gray — цвет мигающего светодиода, когда камера готова читать QR (p000/hl5.java)
<code>device_info.serial_number</code> , <code>device_info.mac</code>	string null

Логика выбора (p000/nb8.java:159–192): `qr` в `methods` → беспроводной сценарий; `sn` + серийный номер → проводной по серийнику; иначе `mac` + MAC → проводной по MAC. Если возможны оба — пользователь выбирает. Ошибки: `UNRECOGNIZED_FORMAT`, `UNSUPPORTED_DEVICE`.

Локального каталога моделей в приложении нет — все данные о моделях приходят отсюда.

9.4. Проводная привязка

Тот же `POST /servers?op=CREATE`, но `device_id` = серийный номер или MAC и соответствующий `device_id_type` (p000/a5a.java:84–96). Пользователь перезагружает камеру по питанию с подключённым Ethernet; дальше — тот же опрос, что в 9.2.

9.5. Перевод привязанной камеры на Wi-Fi

Только для **уже привязанной и находящейся онлайн** камеры. `serverId` — часть id камеры до : (p000/vqa.java:30–35); отсюда формат id камеры: `<serverId>:<индекс канала>`.

```
POST /servers/{serverId}/wifi?op=SCAN # без тела; сканирует сама камера
→ {"networks": [{"ssid": "...", "protection": "OPEN"|"WPA2"|"WPA"|"WEP", "signal": <int>}]}
```

```
POST /servers/{serverId}/wifi?op=CONNECT
{"ssid": "...", "password": "...", "protection": "OPEN"|"WEP"|"WPA2"}
```

Источники: p000/lqa.java, kqa.java, tqa.java, wb2.java, io1.java:31. Последовательность (p000/ui1.java:180–240, частично декомпилировано — условия цикла приблизительны):

1. `POST /servers/{id}?op=GET` с проекцией `{connected, network_type, wifi:{ssid}}; network_type ∈ eth, wifi, mobile, unknown`.
2. `CONNECT`.
3. Пауза 5 с, затем тот же `GET` каждые 5 с до `connected && network_type == "wifi" && wifi.ssid == <целевой>`; три подряд «не подключена» → неудача; `SERVER_OFFLINE` → «связь потеряна».

9.6. Локальная сеть: что искали и чего нет

Поиск по всему `sources/`: `NsdManager`, `MulticastSocket`, `DatagramSocket`, `DatagramPacket`, `WifiNetworkSpecifier`, `WifiNetworkSuggestion`, классы `Bluetooth`, `WpsInfo`, `bindProcessToNetwork`, литералы `SSDP/mDNS/ONVIF`, `rtsp://`, частные IP-адреса — **ноль совпадений** (при этом `WifiManager` находится — в p000/fx1.java, то есть поиск рабочий). `java.net.Socket` встречается только во внутренностях `OkHttp`. В манифесте нет разрешений `Bluetooth`, `CHANGE_WIFI_MULTICAST_STATE`, `NEARBY_WIFI_DEVICES`. `WifiManager` используется лишь для списка сетей и текущего SSID — к сети камеры приложение не подключается; шаблонов SSID камер нет.

Это закрывает вопрос Тгас #31 со стороны приложения: **прямого LAN/P2P-канала «телефон — камера» нет**. Открывает ли прошивка Cute 2 какие-то локальные порты (RTSP/ONVIF/HTTP) — отдельный вопрос, решается только сканом самой камеры (Тгас #32).

9.7. Что из APK узнать нельзя

- содержимое QR-кода и смысл `text_template` — декодировать реальный QR и посмотреть живой ответ `servers?op=CREATE`;
- протокол «камера ↔ облако» — он в прошивке;
- принимает ли Cute 2 привязку по `sn/mac` — зависит от ответа `RECOGNIZE` на её наклейку.

9.8. Прочие перечисления

- `DeviceType` (`com/ivideon/sdk/network/data/p012v5/DeviceType.java`): `desktop`, `camera`, `dvr`, `doorbell`, `bridge`, `cloud_bridge`, `unknown`.
- Форм-фактор (`p000/yс1.java`, только для UI): `home`, `bullet`, `cupola`.

10. Ошибки

Теги: #ошибки #коды_ошибок #HTTP401 #диагностика

Как API сообщает об ошибках и как их классифицирует приложение. Формат тела и коды — **[код]. Какой HTTP-статус сервер шлёт с каким кодом — из клиента не определить**: ни один класс ошибки не привязан к статусу (кроме 401), статус просто сохраняется.

10.1. Формат ошибки API5

Ошибкой считается **любой не-2xx ответ**. Тело разбирается как ошибка, только если Content-Type: application/json (p000/e21.java:385-412, p000/wm3.java:20-21). Разбор — p000/C0334eu.java:19-105, модель Api5Error — p000/C2382pr.java.

```
{ "success": false, "code": "CAMERA_OFFLINE", "message": "...", "details": {...} }
```

поле	тип	обязательность
success	boolean, строго false	да
code	string	да
message	string	да
details	object	нет. Единственный ключ, который читает клиент, — retry_at (Unix-секунды, дробное) при TOO_MANY_LOGIN_ATTEMPTS

Ответ 2xx с "success": false клиент считает сбоем протокола (Unexpected unsuccessful API5 response).

10.2. Алгоритм классификации

p000/C0297du.java:38-73:

- 1. **HTTP 401** → **AuthError**, независимо от code. Это сигнал «токен недействителен» — запускает обновление токена (см. 03-auth.md). У AuthError есть признаки по тексту message: "Session's subject doesn't exist." и "Incorrect login or password".
- 2. **TOO_MANY_LOGIN_ATTEMPTS** → отдельный класс с details.retry_at.
- 3. Поиск code в статической таблице (10.3).
- 4. Поиск в таблице расширений (p000/xw8.java, только сервис site-security).
- 5. Иначе — GenericError(httpStatus, code).

Рекомендация для своего клиента: ветвиться по code, статус использовать только для 401 и как запасной признак.

10.3. Коды ошибок сервера

Классы — в com/ivideon/sdk/network/data/error/. Смысл выведен из имён — **[вывод]**, если не указано иное.

Общие

code	класс	смысл
BAD_REQUEST	BadRequestError	некорректный запрос
OPERATION_RESTRICTED	OperationRestrictedError	операция запрещена для аккаунта/тарифа
FEATURE_NOT_SUPPORTED	FeatureNotSupportedError	устройство/тариф не поддерживает функцию
INACTIVE_SERVICE	InactiveServiceError	платная услуга не активна
LIMIT_EXCEEDED	LimitExceededError	превышен лимит

Устройства

code	класс	смысл
SERVER_NOT_FOUND / SERVER_OFFLINE	ServerNotFoundError / ServerOfflineError	устройство не найдено / не на связи
CAMERA_NOT_FOUND / CAMERA_OFFLINE	CameraNotFoundError / CameraOfflineError	камера не найдена / не на связи
DEVICE_TIMED_OUT	DeviceTimedOutError	устройство не ответило на команду
BAD_DEVICE_CREDENTIALS	BadDeviceCredentialsError	неверные логин/пароль самого устройства
DEVICE_NOT_ALLOWED	DeviceNotAllowedError	
VENDOR_NOT_FOUND	VendorNotFoundError	
UNRECOGNIZED_FORMAT / UNSUPPORTED_DEVICE	DeviceModelsError.*	ответы /device_models?op=RECOGNIZE
ATTACHMENT_TOKEN_NOT_FOUND / TOKEN_FINISHED	AttachmentToken...Error	привязка камеры
MALFORMED_SCHEDULE	MalformedScheduleError	некорректное расписание

Права доступа

SUBJECT_RIGHT_MISSING_FOR_<X> / SUBJECT_RIGHT_REVOKED_FOR_<X>, где <X> ∈ USER, SERVER, CAMERA, ATTACHMENT_TOKEN, TIMELAPSE, TIMELAPSE_TEMPLATE, EXPORTED_RECORD (для последнего — только MISSING).
Классы SubjectRightMissingFor...Error / SubjectRightRevokedFor...Error.

Совместный доступ: GRANTEE_NOT_FOUND, GRANTEE_LIMIT_EXCEEDED, GRANTEE_DEMO_RESTRICTION, SAME_ACCOUNT (доступ самому себе), IMPLICITLY_SHARED, PERMISSION_NOT_FOUND, PERMISSION_GRANT_NOT_FOUND.

Архив, экспорт, таймлапсы

EMPTY_INTERVAL (в интервале нет архива), RECORD_ALREADY_EXISTS, RECORD_QUOTA_EXHAUSTED, EXPORTED_RECORD_NOT_FOUND, EXPORTED_RECORD_NOT_READY, TIMELAPSE_NOT_FOUND, TIMELAPSE_TEMPLATE_NOT_FOUND.

Вход, регистрация, 2FA

code	класс
TOO_MANY_LOGIN_ATTEMPTS	TooManyLoginAttemptsError (+ details.retry_at)

code	класс
USER_NOT_FOUND, USER_EXISTS	UserNotFoundError, UsersExistsError
PASSWORD_MISMATCH, VULNERABLE_PASSWORD	PasswordMismatch, PasswordVulnerabilityError
SIGN_UP_REQUIRED / SIGNUP_REQUIRED, OTP_MISMATCH, OTP_LOCKED, PROVIDER_ERROR	семейство OtpError (вход по одноразовому коду)
PHONE_NUMBER_USAGE_LIMIT_REACHED	PhoneNumberUsageLimitReached
CHALLENGE_NOT_FOUND, CHALLENGE_EXPIRED, TWO_FACTOR_METHOD_NOT_FOUND, TWO_FACTOR_METHOD_NOT_SELECTED, TOO_FREQUENT, INVALID_CODE, NO_ATTEMPTS_LEFT	TwoFaError.*
BAD_CODE	BadCodeError
EXT_AUTH_ERROR	ExternalAuthError

10.4. Ошибки OAuth-эндпоинта

/auth/oauth/token отвечает **не** конвертом API5, а в стиле OAuth2 (p000/ua0.java):

```
{"error": "invalid_grant", "error_description": "...", "reason": "...", "retry_in": 30}
```

условие	результат (p000/va0.java)
error=invalid_request, reason=T00_MANY_LOG_IN_ATTEMPTS	TwoFaError.TooManyLoginAttemptsError с retry_in
error=invalid_grant, reason=2FA_IS_MANDATORY	TwoFaError.TfaIsMandatory
всё прочее	AuthError

Требование второго фактора — **не ошибка**: это ответ 200 с "proceed_with_2fa": true, см. 03-auth.md.

10.5. Ошибки, которые клиент порождает сам

ситуация	ошибка	источник
не-JSON тело, HTTP 403	AccessDeniedError(status, "ACCESS_DENIED")	e21.java:423–436
не-JSON тело, HTTP 404	NotFoundError(status, "NOT_FOUND")	там же
прочее не-JSON тело или JSON, не прошедший проверку типов	GenericError(status, "UNEXPECTED_ERROR_" + hex(hash(тела)))	wm3.java:31–33
битый JSON в ошибке / в ответе 200	JsonMalformedError	e21.java:412–418, 470–475
любое исключение (сеть, TLS, таймаут)	ExceptionError, статус 0	e21.java:476–482
вызов отменён	networkcall/CallCanceledError, статус 0	

В строках 423–436 jadx вывел условия `if (i != 403)` инвертированными (метод помечен Code duplicated); таблица даёт очевидно задуманное прочтение.

10.6. Устаревший API4

Основной сервис его не использует, но адаптер в приложении остался. Конверт: `{"success": bool, "response": ...}`; при ошибке `response = {code: int, code_alias: string, message, details}` (`p000/C2233lr.java`, `p000/C0258cr.java`). Числовые коды (`p000/C2196kr.java:17`): `10002` → `AuthError`, `10004` → `NotFoundError`, `10005` → `AlreadyExistsError`, `10008` → `RecursiveGrantingError`, `10020` → `InactiveServiceError`, `20001` → `OperationRestrictedError`, `20003` → `LimitExceededError`.

10.7. Рекомендации для своего клиента

- Сначала смотрите HTTP-статус: 2xx → конверт успеха; иначе попробуйте разобрать `{success, code, message}`; не получилось — сохраняйте статус и кусок тела.
- 401 → один раз обновить токен и повторить запрос; повторный 401 → требовать новый вход.
- `CAMERA_OFFLINE` / `SERVER_OFFLINE` — штатная ситуация, а не сбой: показывайте «камера не в сети».
- `TOO_MANY_LOGIN_ATTEMPTS` / `retry_at`, `retry_in`, `TOO_FREQUENT` — **уважайте паузу**, не повторяйте вход в цикле.
- Неизвестные `code` обрабатывайте обобщённо — таблица отражает только то, что знала версия 3.7.1.

11. Карта кода

Теги: #карта_кода #p000 #обфускация #jadx

Где что лежит в `decompiled-ivideon-3.7.1/sources/`. Имена вида `C2199ku` стабильны для **jadx 1.5.6 с `--deobf`**; в скобках — настоящее имя класса в DEX (из комментария `renamed from`), оно не зависит от версии jadx. Если у класса есть «говорящий» `toString()`, оттуда взято исходное имя.

Как читать такие классы — [01-methodology.md](#), раздел 1.5.

11.1. Сервисные интерфейсы (точки входа)

файл	содержимое
p000/InterfaceC2236lu.java (lu)	основной сервис API5, ~75 методов
p000/InterfaceC2345or.java (or)	картинки: <code>live_preview</code> , миниатюры, загрузка по URL
p000/jc0.java	<code>POST /auth/oauth/token</code>
p000/tl7.java	публичные: шаги 2FA (динамический URL), регистрация, сброс пароля, конфиг формы
p000/qm2.java	<code>/customer_clouds?op= FIND</code>
p000/cv4.java	<code>/idam?op= SHARE</code> — доступ к камере по номеру телефона
p000/pp0.java, p000/vx8.java, p000/e26.java	баннеры, site security, маркетинг

11.2. Ядро сети и сессии

файл	роль
p000/i85.java	ядро SDK: OkHttpClient (76–92), Gson и адаптеры (91–120), токен, проверка срока (218–240), блокировка обновления (243–295)
p000/k85.java	сборка i85, выбор хостов по режиму (47–151)
p000/l21.java	константы: хосты, бренды, права для шаринга, email поддержки
p000/yl6.java	набор сервисов на токен; создаёт C2199ku с api_host (28)
p000/or7.java	нормализация api_host → <code>https://...</code> (167–171)
p000/C2199ku.java (ku)	«сессия API»: base URL, сервисы, интерцептор; m14125h (152) — получение подписанного URL без выполнения запроса
p000/zn3.java	фабрика Retrofit-сервисов
p000/ek6.java	<code>NetworkCall<T></code> — обёртка вызова
p000/e21.java	выполнение: повторы (81–143), 401 → refresh (86–103), разбор ошибок (385–482)
p000/g58.java → mo7583n (396–467)	интерцептор авторизации и подписи
p000/zbc.java → m22152b (40–43)	путь для подписи: <code>"/" + join(pathSegments)</code>
p000/C2504t.java → case 1 (119–138)	<code>signWithSecret</code> : HMAC-SHA1
p000/kr6.java → kr6(C2504t) (626–647)	генерация 8-символьного id сессии

файл	роль
p000/ps4.java, e82.java, g08.java, a40.java, fo4.java, v28.java	OkHttp: <code>HttpUrl</code> , <code>HttpUrl.Builder</code> , <code>Request</code> , <code>Request.Builder</code> , <code>Headers</code> , <code>ResponseBody</code>
p000/qs6.java, p000/ns7.java	OkHttp: <code>OkHttpClient.Builder</code> , <code>BridgeInterceptor</code> (User-Agent <code>okhttp/5.3.2</code>)

11.3. Авторизация

файл	роль
p000/qv5.java → qv5(i85, ry1) (697–701), m17621g (358–366)	форма OAuth: поля клиента/устройства, <code>grant_type</code> , <code>trusted_device</code>
p000/C2552ua.java (63–81)	Authorization: Basic <code>base64("android-client:")</code>
p000/fr7.java, p000/C2470s2.java (s2)	модель токена; C2470s2 добавляет флаг наличия <code>hmac_secret</code> и подписчик
p000/h85.java	конечный автомат обновления токена (лог-строки <code>Update token: ...</code>)
p000/ob0.java	хранение токена в <code>SharedPreferences</code> , состояние входа
p000/ic0.java	вход, 2FA (<code>SELECT 645</code> , <code>RESEND 581</code> , <code>SUBMIT 715</code>), выход, отзыв (180–183, 345–348)
p000/sl7.java (23–34)	сборка динамических URL 2FA
p000/j5a.java, bb0.java, eb0.java, b18.java, j99.java, ta0.java	DTO 2FA: вызов, метод, состояние, тела запросов
p000/ua0.java, p000/va0.java	тело и классификация ошибок OAuth
p000/C2270mr.java	адаптер ответа токена / вызова 2FA (не декомпилируется, нужен fallback-режим)
p000/ry1.java, p52.java, q10.java	идентификаторы устройства: <code>device_type</code> , <code>device_name</code> , Firebase Installation ID
p000/xt8.java	тело регистрации
p000/C2582v2.java	десериализатор ресурса access-token

11.4. Формат данных и ошибки

файл	роль
p000/C2619w2.java	адаптеры: конверт успеха (64–114), распаковка <code>items</code> (131–143), словарь-по-id (144–165)
p000/ld3.java (364–380)	фабрика конвертеров; обработка <code>@kr7</code>
p000/nr7.java	«сырой» JSON-результат для <code>@kr7</code>
p000/nq2.java, p000/C2647wu.java	даты: чтение (секунды/мс), запись (секунды с дробью)
p000/un6.java, p000/sn6.java	пустой результат
p000/ud7.java	<code>turned_off_until</code>
p000/z10.java	дата <code>yyyy-MM-dd</code>

файл	роль
p000/C0334eu.java, p000/C2382pr.java	разбор и модель Api5Error
p000/C0297du.java (19, 38–73)	таблица code → класс ошибки, правило 401
p000/xw8.java	таблица ошибок site security
p000/wm3.java	ошибки для не-JSON ответов
com/ivideon/sdk/network/data/error/	иерархия ошибок (имена сохранены), базовый NetworkError
p000/C2233lr.java, C0258cr.java, C2196kr.java; com/ivideon/sdk/network/service/p013v4/	наследие API4

Аннотации: wy6 POST, mg4 GET, g47 Path, fp7 Query, ey0 Body, iba Url, xa4 FormUrlEncoded, zz3 FieldMap, bo4 HeaderMap, ok8 SerializedName, ka5 JsonAdapter, mn6 без токена, zc0 Bearer, l85 подпись, kr7 сырой результат.

Gson: jm4 Gson, km4 GsonBuilder, o5a TypeAdapter, p5a TypeAdapterFactory, x6a TypeToken, jc5/bd5 JsonReader/Writer, zb5 JsonObject, pa5 JsonArray, fb5 JsonElement.

11.5. DTO по областям

область	файлы (p000/)
серверы и камеры	vk8 ServerSlice, tm1 CameraSlice, r23 репозиторий + проекция (64–99), p14 тело FIND, kj7 тело с проекцией, ug1 права, xzb разбор services, com/ivideon/sdk/network/data/p012v5/DeviceType
привязка	gl2 тело servers CREATE, ja0 токен привязки, ka0 статус, u13 тип id, zh4 тело QR, rw2 тело DELETE, vs7/w13/w96/v13 RECOGNIZE, ia0 способы, hl5 цвет LED, es0 правило & для Cute 2
привязка — логика	w03, q91, p91, l91, C2138j7, C2175k7, js8, ls8, a5a, nb8, m13, C2711yj (не декомпилируется)
Wi-Fi	lqa/kqa/tqa скан, wb2 CONNECT, io1, ui1, ti1, vqa, fx1 (WifiManager телефона)
управление	ql7/pl7/ol7 PTZ, db7/cb7/y65/za6 плагины, ab6/va6/bb6 движение, m19/l19/n19 звук, il5/gl5 LED, x65/w65 ИК, k86 микрофон, zaa config, qz1 режим архива, ku7 расписание, bg8/cg8/zf8 SD-карта, cba, bba
видео	sw4 качество, ct5 LivePreviewChannel, mo7/lo7 голос
архив	fo9/g50/o50/n30 таймлайн, yh4/xh4/d20/a20/e20/c20 календарь, t20/уу3/cv3/bv3 экспорт
таймлапсы	to9, np9, sr9, kt9, ot9
события	tp3 фильтр, en3 Event, ho3 источник, qo3/oo3 типы, gp6/hp6/nd1/o14 настройки уведомлений
пользователь, доступ	ada User, gda, hi4, vw2, r65, ro8 SHARE, ci4, dl2/la2 2FA-методы, qo8 IDAM
пакеты	fv0, gv0, yb7, cv0 сборщик, zb7 интервал, b77, C2340on цикл (не декомпилируется)
облака	xz1 CustomerCloud, i86/te2 microcloud
push	ho7 тело подписки, bo7 провайдеры, xn7, pc7, nc1, jrc; com/ivideon/client/notifications/push/

11.6. Плеер и медиа

файл	роль
<code>com/ivideon/sdk/player/p014iv/core/IvPlayer.java</code>	JNI-обёртка: native-методы, события (106–141)
<code>com/ivideon/client/p009ui/player/PlayerController.java</code>	экран плеера (190 КБ): сборка URL (785–837), перемотка (1873–1900), скорости (2710), токен-обсервер (2276–2293)
<code>p000/kr2.java</code> (84–102)	<code>nativeSetMedia</code> : live / архив
<code>p000/t75.java</code> , <code>r75.java</code> , <code>q75.java</code> , <code>p75.java</code>	управление плеером, поверхность, трансформации
<code>p000/g75...n75.java</code>	классы событий плеера
<code>p000/fr2.java</code> (323–400)	обработка событий, переключение
<code>p000/d97.java</code>	конечный автомат UI плеера
<code>p000/e97.java</code> (41–73)	расчёт текущего времени на шкале
<code>p000/C2363p8.java</code> , <code>C2210l4.java</code>	скорость воспроизведения
<code>p000/ca1.java</code> , <code>yh1.java</code>	загрузка кадра-превью
<code>p000/vd1.java</code> , <code>wd1.java</code> , <code>et5.java</code> , <code>dt5.java</code> , <code>k06.java</code> , <code>r66.java</code> , <code>mg7.java</code>	WebSocket-превью: запрос, сокет, protobuf
<code>p000/you8.java</code> , <code>ip3.java</code>	клипы событий через <code>MediaPlayer</code>
<code>p000/du3.java</code> , <code>to7.java</code> , <code>lw6.java</code> , <code>zoa.java</code> , <code>yoa.java</code> , <code>bpa.java</code> ; <code>com/ivideon/tools/opus/internal/LibOpus.java</code>	голос: режим, захват, Opus, потоковый POST, лимиты; <code>apa</code> , <code>fm8</code> — не декомпилируются

Нативные библиотеки — в `apk/xapk/config.arm64_v8a.apk`, `lib/arm64-v8a/` (в `decompiled-.../resources` их нет: это отдельный сплит).

11.7. UI, не переименованный R8

путь	содержимое
<code>com/ivideon/client/p009ui/wizard/</code>	мастер подключения: <code>methods/p010qr</code> , <code>methods/wired</code> , <code>methods/wifi</code> , <code>methods/desktop</code>
<code>com/ivideon/client/p009ui/player/</code>	плеер
<code>com/ivideon/client/p009ui/cameralayout/</code>	сетка камер
<code>com/ivideon/client/p009ui/signin/</code>	вход, 2FA
<code>com/ivideon/client/notifications/push/</code>	приём push, <code>CloudSubscriptionWorker</code>
<code>resources/res/navigation/*.xml</code>	графы навигации — удобная «карта сценариев»
<code>resources/res/values/strings.xml</code>	тексты; имена ресурсов часто раскрывают назначение кода
<code>resources/res/xml/network_security_config.xml</code>	политика TLS
<code>resources/AndroidManifest.xml</code>	компоненты, разрешения, deep links

11.8. Методы, которые jadx не декомпилировал

Для них нужен `jadx --single-class <имя> -m fallback` (вывод ближе к байткоду, но полный) либо `jadx-gui / smali`:

класс	что в нём
C2199ku.m14118a	составной запрос (по сигнатуре — выборка по набору id за интервал)
C2711yj (case 11)	запрос QR-кода привязки
C2270mr	адаптер токена / вызова 2FA, адаптер API4
C2340on	цикл опроса пакетного запроса
apa, fm8	цикл кодирования Opus и запись чанков
go1	чтение сетевой информации камеры
ui1 (частично)	цикл переподключения Wi-Fi

Fallback-дампы, сделанные при подготовке этой документации, **в проект не сохранены** (лежали во временном каталоге сессии). При необходимости повторите команду.

12. Клиент на Zig (zig-client/)

Теги: #Zig #клиент #Windows #GDIplus #подпись_URL

Собственная программа проекта: консольный клиент под Windows, который входит в аккаунт Ivideon, находит камеру **Ivideon Cute 2** и показывает её изображение. Язык и цель выбраны пользователем (Trac #38): Zig 0.16, без сторонних библиотек.

12.1. Состояние на 2026-09-20

часть	файл	состояние
сборка	zig-client/build.zig	готово: цели <code>install</code> , <code>run</code> , <code>test</code>
подпись URL	zig-client/src/sign.zig	готово и покрыто тестами (4 теста, векторы из 05-url-signing.md)
точка входа	zig-client/src/main.zig	заглушка: пустой <code>main</code> , подключает тесты <code>sign.zig</code>
HTTP/конверт ответа	src/api.zig	не написано (Trac #53)
OAuth, файл токена	src/auth.zig	не написано (Trac #40)
окно просмотра	src/viewer_win32.zig	не написано (Trac #54)

```
cd zig-client
..\tools\zig\zig.exe build test      :: тесты подписи; молчание и код 0 = успех
..\tools\zig\zig.exe build          :: zig-out\bin\ivideon.exe
```

12.2. Почему облако, а не USB или LAN

- **USB** (Trac #51, проверено на машине пользователя): Cute 2 по USB не определяется никак — ни UVC-камера, ни сетевой адаптер RNDIS/ECM, ни COM-порт, ни диск; её нет и в истории USB-устройств Windows. Micro-USB у камеры — только питание.
- **LAN** (Trac #32, не завершено): скан локальной сети не удался — AmneziaVPN на ПК заворачивает 192.168.0.0/24 в туннель. Отдаёт ли прошивка RTSP/ONVIF/HTTP локально — неизвестно. В коде Android-приложения прямого обмена с камерой по LAN не найдено (см. [09-camera-attachment.md](#)).
- **Облако**: протокол восстановлен — этим путём идёт программа. Условие: камера привязана к аккаунту и в сети по Wi-Fi.

12.3. Команды (проект)

команда	действие	запрос API
ivideon login	спрашивает email и пароль (пароль без эха; либо <code>IVIDEON_EMAIL</code> / <code>IVIDEON_PASSWORD</code>), сохраняет токен в <code>.ivideon/token.json</code> . Пароль не сохраняется	POST <code>/auth/oauth/token</code>
ivideon cameras	серверы и камеры: id, имя, online, модель, прошивка	POST <code>{api_host}/servers?op= FIND</code>
ivideon snapshot [camera_id] [файл.jpg]	один кадр в файл	GET <code>/cameras/{id}/live_preview</code> (подписанный)

команда	действие	запрос API
<code>ivideon watch</code> <code>[camera_id] [--fps N] [--q 0..2]</code>	окно с «живой картинкой» из JPEG-превью	тот же <code>live_preview</code> в цикле
<code>ivideon url</code> <code>[camera_id] [--q 0..2]</code>	печатает подписанный URL живого видео для VLC/ffplay/mpv. URL содержит токен	<code>GET</code> <code>/cameras/{id}/live_stream</code>
<code>ivideon logout</code>	удаляет локальный файл токена (отзыв на сервере — <code>POST /access_tokens/{id}?op=REVOKE</code> — возможное улучшение, см. 03-auth.md)	—

У всех команд планируется ключ `--raw` — печать ответа сервера с вырезанными секретами, для первой живой сверки протокола.

12.4. Модули

файл	ответственность
<code>src/main.zig</code>	разбор командной строки, диспетчер команд, человекочитаемые ошибки
<code>src/api.zig</code>	HTTP через <code>std.http.Client</code> (TLS — реализация Zig std); конверт ответа; добавление <code>access_token</code> ; повтор запроса после обновления токена
<code>src/auth.zig</code>	OAuth-запрос (form-urlencoded + <code>Basic base64("android-client:")</code>), модель токена, чтение/запись <code>.ivideon/token.json</code> , <code>refresh_token</code>
<code>src/sign.zig</code>	<code>signedUrl()</code> — сборка query в кодировке OkHttp, <code>cseq</code> , <code>cs</code>
<code>src/viewer_win32.zig</code>	окно Win32; JPEG декодирует GDI+ (<code>gdiplus.dll</code> , поток из памяти — <code>SHCreateMemStream</code> из <code>shlwapi.dll</code>); перерисовка по таймеру; сеть в отдельном потоке

API `sign.zig`

```
const sign = @import("sign.zig");

const url = try sign.signedUrl(gpa, .{
    .base_url    = "https://eu01-api.ivideon.com",    // без завершающего '/'
    .path        = "/cameras/<id>/live_stream",
    .params      = &.{ .{ .name = "q", .value = "2" } },
    .access_token = token.access_token,
    .hmac_secret = token.hmac_secret,                // "" → URL без подписи
    .session     = session,                          // 8 символов [0-9a-zA-Z], один на сессию
    .counter     = monotonic_ms,                     // не уменьшать в пределах session
});
defer gpa.free(url);
```

`signedUrl` возвращает `error.UnsupportedPathCharacter`, если в `path` есть символ, который OkHttp закодировал бы: в этом случае подписываемый (декодированный) путь и путь в URL различались бы, а такой сценарий для реальных id камер не нужен и не реализован.

12.5. Проектные решения

- **JPEG-превью вместо декодирования видео.** Живой поток — H.264/H.265; декодера на чистом Zig нет, а FFmpeg ради первой картинки — лишняя зависимость. `live_preview` отдаёт JPEG, который Windows

декодирует сама. Это несколько кадров в секунду, не полноценное видео.

- **Полноценное видео — внешним плеером.** Приложение Ivideon поступает так же: строит подписанный URL и отдаёт строку плееру на FFmpeg. Команда `url` даёт ту же строку.
- **Только std и системные DLL.** Сборка одной командой, без менеджера пакетов.
- **Секреты.** Пароль не хранится и не печатается; токен — в локальном файле, который должен попасть в `.gitignore`; URL с токеном выводится только по явной команде `url`.

12.6. Где вероятнее всего сломается при первом запуске

1. **TLS.** `std.http.Client` использует собственную реализацию TLS из Zig std; совместимость с серверами Ivideon (набор шифров, цепочка сертификатов) не проверена.
2. **Редирект на региональный хост.** `api.ivideon.com` отвечает редиректом на `eu01-api.ivideon.com` (проверено для корня). POST с телом при 301/302 многие клиенты превращают в GET — редирект логина надо обрабатывать вручную, повторяя тот же POST на `Location`. После входа все запросы идут сразу на `https://{api_host}` из токена, и проблема исчезает.
3. **Форма ответа `/servers?op= FIND`.** Метод помечен `@kr7` — приложение получает «сырой» JSON и разбирает его вручную, поэтому структура известна хуже, чем у типизированных ответов. См. [06-api-reference.md](#).
4. **2FA.** Если на аккаунте включена двухфакторная авторизация, придёт токен с `limited_to_2fa: true`. Первая версия должна это распознать и сообщить, а не падать. См. [03-auth.md](#).
5. **Content-Type у `live_preview`.** Ожидается `image/jpeg`; не проверено.

12.7. Порядок работы (Trac)

1. **52 Zig 0.16.0 в `tools/zig` — сделано.**
2. **53 каркас CLI + `sign.zig` с тестами — `sign.zig` готов, CLI нет.**
3. **40 `login` → #41 `cameras` → #42 `snapshot, url` — первая живая проверка протокола; запускает пользователь со своим аккаунтом.**
4. **54 `watch` — окно просмотра.**

13. Открытые вопросы и план живой проверки

Теги: #открытые_вопросы #живая_проверка #риски #безопасность

Вся документация построена на **статическом анализе**. Этот раздел отделяет известное от неизвестного и описывает, в каком порядке проверять протокол на реальном аккаунте, не навредив ни аккаунту, ни камере.

13.1. Что проверено фактически

утверждение	как
api.ivideon.com редиректит на eu01-api.ivideon.com; openapi-alpha.ivideon.com → openapi-alpha-eu01.ivideon.com	запрос без авторизации
Cute 2 по USB не определяется в Windows никак (только питание)	осмотр PnP-устройств (Trac #51)
реализации подписи на Zig и Python совпадают на трёх векторах	zig build test, прогон кода из 05-url-signing.md

Всё остальное — чтение кода клиента.

13.2. Вопросы, которые закроет первый живой запуск

№	вопрос	где описано	как проверить
1	Принимает ли сервер вход от стороннего клиента (User-Agent, device_instance_id)?	03	login
2	Каким кодом редиректит POST /auth/oauth/token и сохраняется ли тело?	04 §4.7	login, смотреть 3xx
3	Значение api_host и наличие hmac_secret в токене	03	login --raw (секреты замаскированы)
4	Форма result у /servers?op=FIND: массив, {items} или словарь по id	04 §4.4	cameras --raw
5	device_model, software_version, features у Cute 2	06 §6.1	cameras
6	Принимает ли сервер нашу подпись cs; Content-Type у live_preview	05, 07	snapshot
7	Открывается ли live_stream в ffplay/VLC/mpv; контейнер, кодеки, сигнализация HEVC	07 §7.3	url → ffprobe
8	Что вернёт сервер без projection	04 §4.5	один запрос
9	Срок жизни подписанного URL; реакция на повтор cseq	05	повторить старый URL

13.3. Вопросы, требующие отдельной работы

вопрос	путь решения	Trac
Смысл аргументов nativeSetMedia, коды ошибок плеера, метаданные FLV (syncTime, fragmentPosition)	дизассемблирование libivplayerjni.so (Ghidra/rizin) либо разбор дампа живого потока	#12, #5

вопрос	путь решения	Trac
Содержимое QR-кода привязки, смысл <code>text_template</code>	декодировать реальный QR; посмотреть ответ <code>servers?op=CREATE</code>	#30
Протокол «камера ↔ облако»	только анализ прошивки/трафика камеры — вне APK	—
Открывает ли Cute 2 локальные порты (RTSP/ONVIF/HTTP)	скан камеры при отключённом VPN (AmneziaVPN заворачивает <code>192.168.0.0/24</code>)	#32
Формат URL WebSocket-превью и голосового канала	живые вызовы <code>live_preview?op=STREAM</code> , <code>voice_message?op=START</code>	#24, #26
Как ограниченный 2FA-токен становится полным	трассировка <code>lw5/ic0</code> или живой сценарий	#15
JSON-форма <code>recording_schedule</code> , формат <code>month</code> в календаре, единицы зоны детектора движения	чтение кастомных сериализаторов / живые ответы с <code>"method": "GET"</code>	#17, #25
Лимиты частоты запросов, условия использования	документы <code>lvideon</code> ; наблюдение кодов <code>TOO_FREQUENT</code> , <code>LIMIT_EXCEEDED</code>	#36
Защита приложения (root-детект, проверка целостности)	пиннинга нет — установлено; остальное не смотрели	#11

13.4. План живой проверки

Принцип: **сначала только чтение**, по одному запросу, с паузами; ничего не менять на камере и в аккаунте, пока не подтверждён базовый протокол.

шаг	действие	риск	критерий успеха
0	убедиться, что камера привязана и показывает картинку в официальном приложении; выяснить, включена ли 2FA	—	—
1	<code>POST /auth/oauth/token</code> — одна попытка	блокировка при частых неудачах (<code>TOO_MANY_LOG_IN_ATTEMPTS</code>) → не повторять в цикле	токен, либо распознанный <code>proceed_with_2fa</code>
2	<code>POST /servers?op=FIN</code> с проекцией из 06	нет	виден сервер Cute 2 и камера <code><serverId>:0</code>
3	<code>GET live_preview</code> (подписанный)	нет	JPEG-файл открывается
4	<code>GET live_stream</code> → <code>ffprobe/ffplay</code> , 10–20 секунд	трафик	определён контейнер и кодеки
5	<code>POST /events?op=FIN</code> за последний час; <code>events?op=GET_TYPES</code>	нет	список событий
6	<code>plugins?op=INVOKE</code> с <code>"method": "GET"</code> для возможностей из <code>features</code>	нет (чтение)	текущие настройки детекторов/LED
7	refresh токена	нет	новый <code>access_token</code>
8	запись — только осознанно : <code>led_switch</code> on/off как самый безобидный тест	обратимо	светодиод переключился

Не выполнять при проверках: `servers DELETE`, `cameras DELETE`, `sd_card FORMAT`, `software_version UPDATE`, `auth_credentials UPDATE`, `wifi CONNECT`, `users DELETE`, `access_tokens REVOKE` для токена официального

приложения.

Безопасность данных: пароль вводить только в свой клиент локально; токены и подписанные URL не вставлять в тикеты, вики и чаты — в `--raw`-выводе маскировать `access_token`, `refresh_token`, `hmac_secret`, `cs`, `challenge_token`.

13.5. Расхождения с ранней версией вики Trac

Найдены при подготовке этой документации; страницу `Protocol` в Trac стоит привести в соответствие.

было в вики	как на самом деле
<code>live_preview</code> авторизуется заголовком Bearer (<code>@zc0</code>)	<code>@l85: access_token</code> в query + подпись; Bearer — только миниатюры и загрузка по готовому URL
видео-URL «открывает FFmpeg»	открывает собственный C++-плеер (Boost.Beast + FLV-демуксер); FFmpeg собран без демуксеров и протоколов
контейнер потока неизвестен	FLV поверх HTTP(S)
использование <code>api_host</code> — гипотеза	<code>api_host</code> — базовый URL всего авторизованного API и видео
хост <code>streaming.ivideon.com</code> — для потоков	не используется в 3.7.1
признак 2FA — <code>limited_to_2fa</code>	обычная 2FA — ответ 200 <code>proceed_with_2fa</code> + <code>challenge_token</code> ; <code>limited_to_2fa</code> — сценарий принудительной настройки 2FA
<code>/idam?op=SHARE</code> — SSO	выдача доступа к камере по номеру телефона
превью-поток — «поток кадров»	WebSocket + protobuf с JPEG